

Network Computing

(058172)



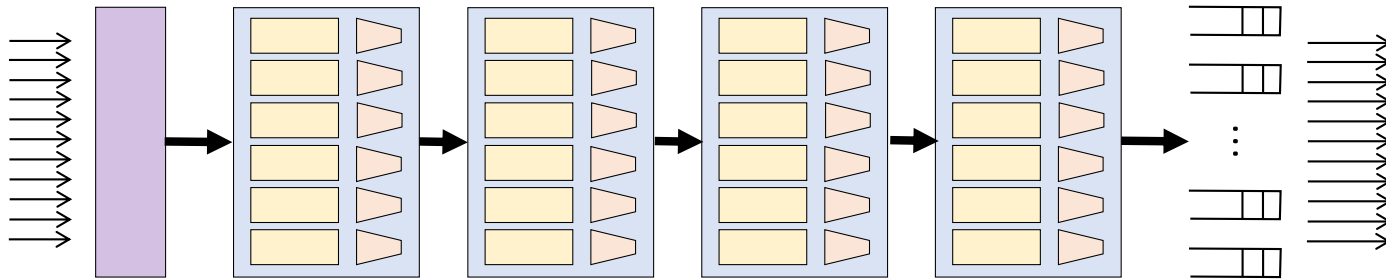
POLITECNICO
MILANO 1863

Data Structures

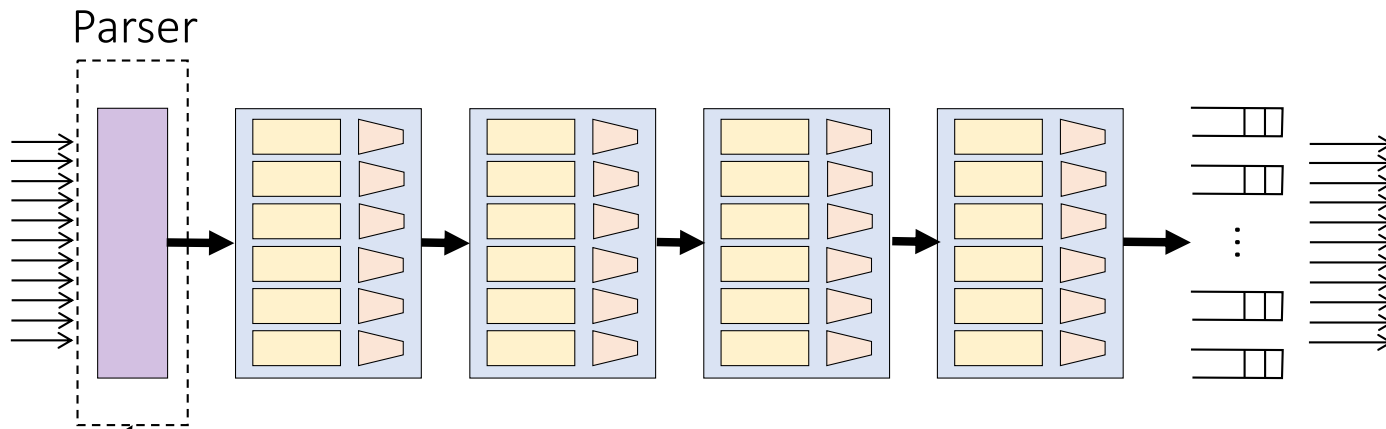
Content

- Ternary Content-Addressable Memory
- Hash
- Bloom Filters
- Invertible and Counting Bloom Filters
- Sketches

Recap from the previous lecture (1)

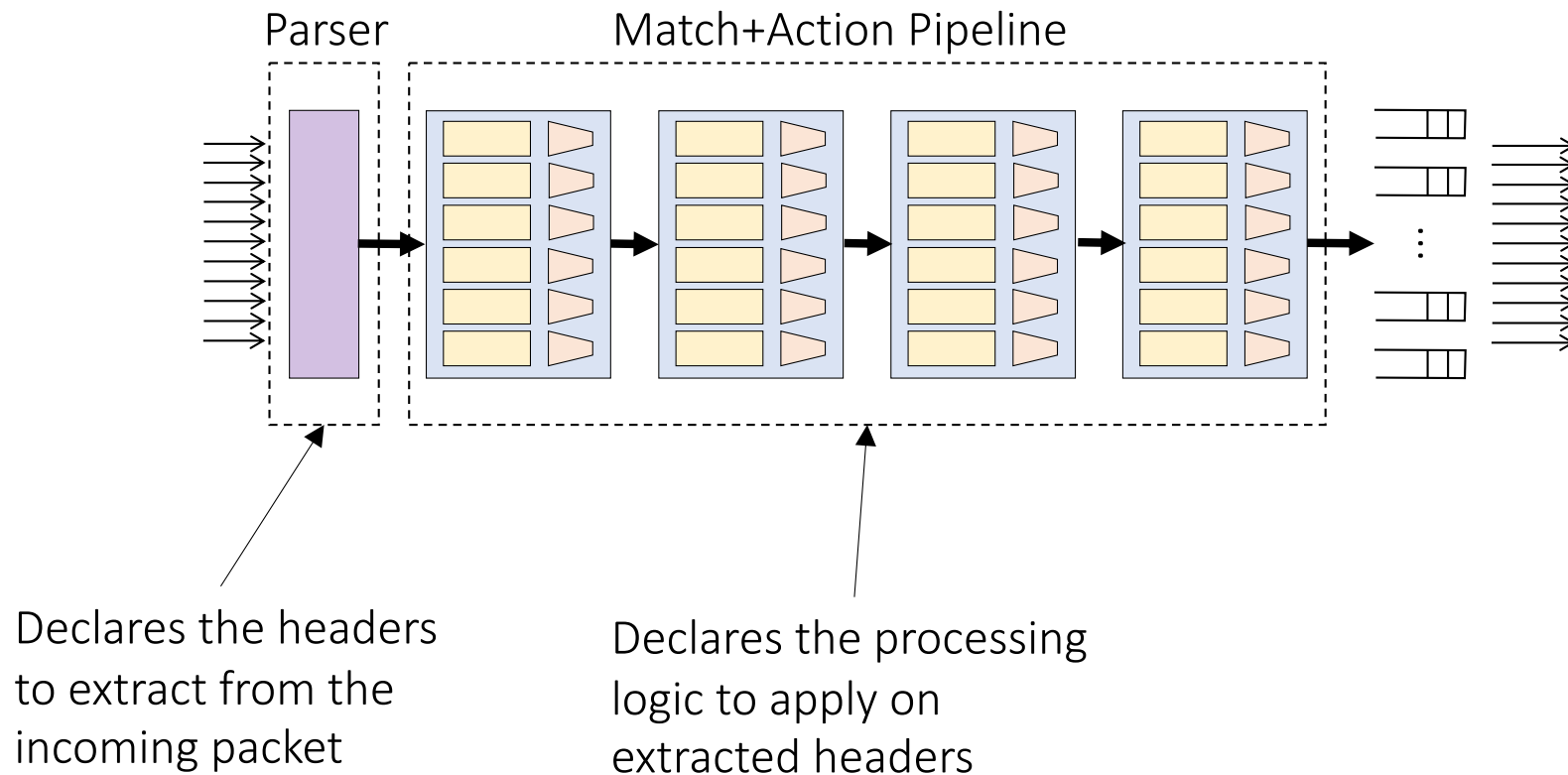


Recap from the previous lecture (2)

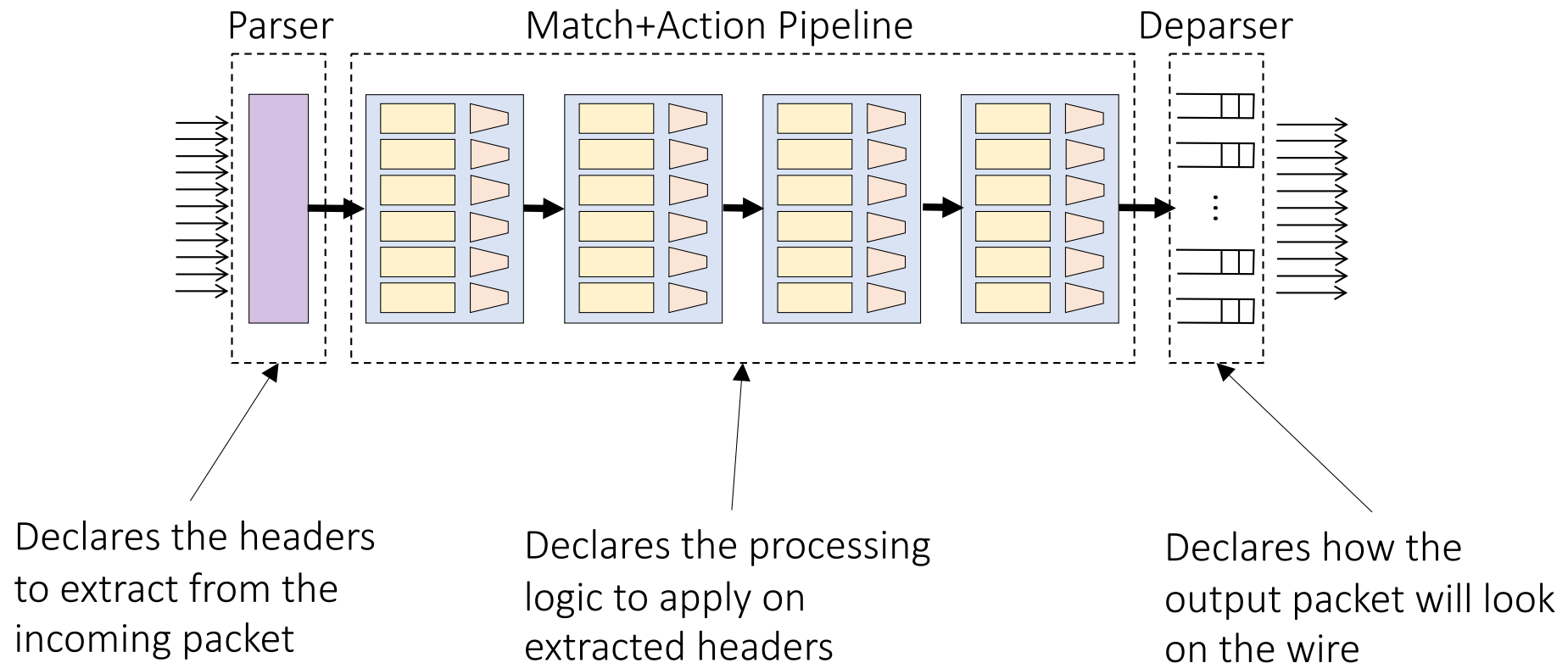


Declares the headers
to extract from the
incoming packet

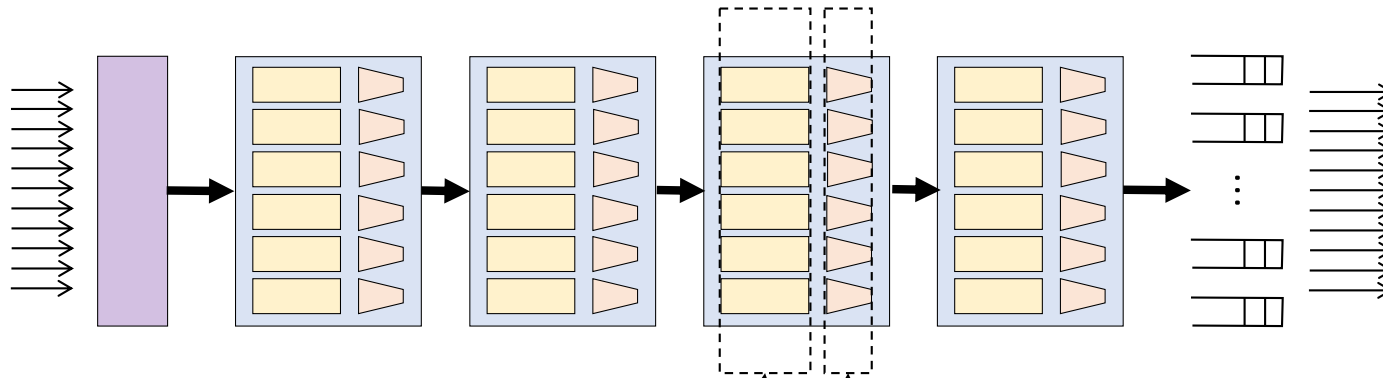
Recap from the previous lecture (3)



Recap from the previous lecture (4)



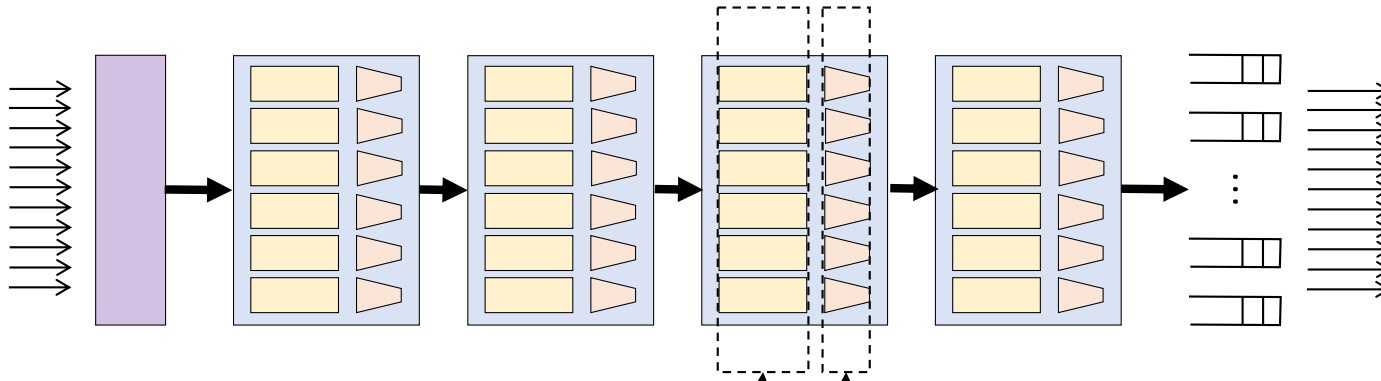
Recap from the previous lecture (5)



Match Logic: a mix of SRAM, TCAM for lookup tables and other things

Action Logic: Arithmetic Logic Units (ALU) for boolean and arithmetic operations + header modification and other things

The IP router lookup example



Match Logic: given the IP destination of the input packet, find the longest prefix match (LPM) in the table

Action Logic: forward the LPM out from port X

How to quickly find the LPM?

- Let's assume we have thousands of entries
- What would you do?
- Linear search?

192.168.2.1 →

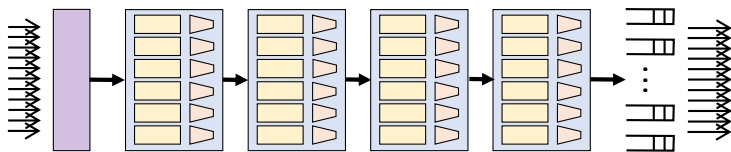
192.168.3.0/24
192.168.1.0/24
192.168.2.0/24
192.168.3.0/28

How to quickly find the LPM?

- Let's assume we have thousands of entries
- What would you do?
- Linear search?

192.168.2.1 →

192.168.3.0/24
192.168.1.0/24
192.168.2.0/24
192.168.3.0/28



Remember:

- Each stage process one packet
- Each stage has to have predictable processing latency so we do not stall the pipeline

TCAM to the rescue

- TCAM stands for Ternary Content Addressable Memory
- It is a special type of memory, called associative
- It compares input data against the stored data and returns the address of matching data

RAM vs TCAM

RAM	
write	read
<i>write X at address Y</i>	<i>What's stored at address X?</i>

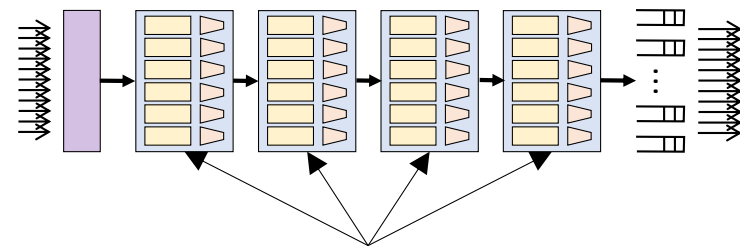
TCAM	
write	read
<i>write X at address Y</i>	<i>Where is X stored?</i>

TCAM memory

- TCAM is called *Ternary* because it allows to store in memory 3 types of symbols: 0, 1 and * (don't care)
- TCAMs produce the result of the search in constant time and because of this are optimal to be used for searching in high-performance-architectures

- Problems

- TCAMs are extremely power hungry
- TCAMs are expensive



Every stage has a bit of TCAM

TCAM in operation (example 1)

- Given the input in the figure, what the TCAM would provide?

The diagram shows a TCAM table with 6 rows. The first three rows contain entries, while the last three are empty. An arrow labeled '192.168.2.1' points to the 'address' column of the fourth row (index 3). Arrows labeled 'address' and 'entry' point to the respective columns of the first row (index 0).

address	entry
0	192.168.3.0/24
1	192.168.1.0/24
2	192.168.2.0/24
3	
4	
5	

TCAM in operation (example 1)

- Given the input in the figure, what the TCAM would provide? **2**

The diagram shows a TCAM table with 6 rows, indexed 0 to 5. The first column is labeled 'address' and the second column is labeled 'entry'. An arrow points from the input '192.168.2.1' to the 'address' column of row 3.

address	entry
0	192.168.3.0/24
1	192.168.1.0/24
2	192.168.2.0/24
3	
4	
5	

TCAM in operation (example 1)

- Given the input in the figure, what the TCAM would provide? **2**
- Is this enough to understand what to do with a packet?

The diagram shows a TCAM table with 6 rows, indexed 0 to 5. The first column is labeled 'address' and the second column is labeled 'entry'. The entries in the first three rows are 192.168.3.0/24, 192.168.1.0/24, and 192.168.2.0/24. The last three rows are empty. An input address 192.168.2.1 is shown with an arrow pointing to the 'address' column of row 3.

address	entry
0	192.168.3.0/24
1	192.168.1.0/24
2	192.168.2.0/24
3	
4	
5	

TCAM in operation (example 1)

- Given the input in the figure, what the TCAM would provide? **2**
- Is this enough to understand what to do with a packet?

TCAM	
0	192.168.3.0/24
1	192.168.1.0/24
2	192.168.2.0/24
3	
4	
5	

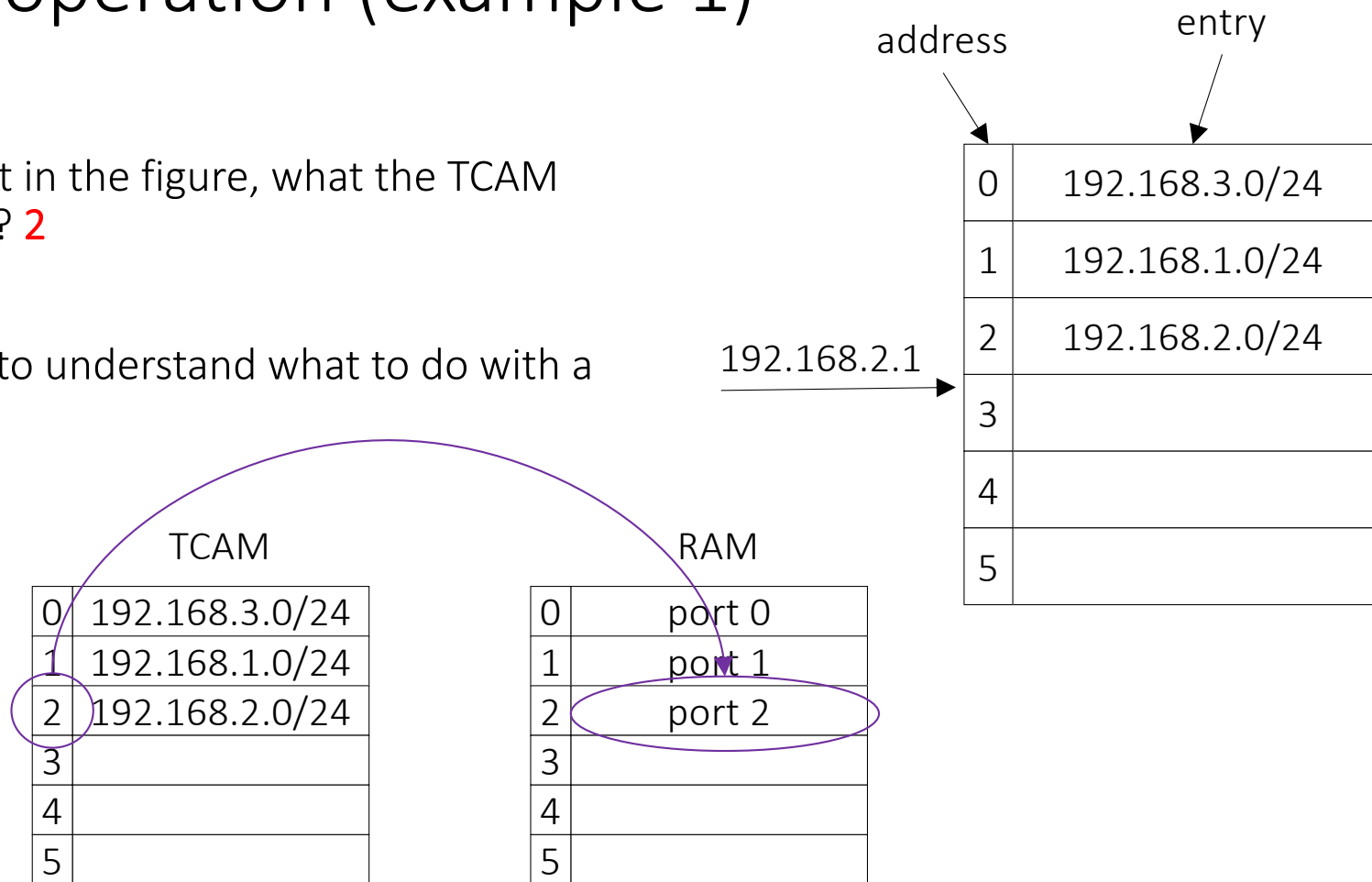
RAM	
0	port 0
1	port 1
2	port 2
3	
4	
5	

address	entry
0	192.168.3.0/24
1	192.168.1.0/24
2	192.168.2.0/24
3	
4	
5	

192.168.2.1 →

TCAM in operation (example 1)

- Given the input in the figure, what the TCAM would provide? **2**
- Is this enough to understand what to do with a packet?



TCAM in operation (example 2)

- Given the input in the figure, what the TCAM would provide?

The diagram shows a TCAM table with 6 entries. An input address '192.168.2.1' is shown with an arrow pointing to the 'address' column of the table. The 'entry' column contains network prefixes. The table is as follows:

address	entry
0	192.168.3.0/24
1	192.168.1.0/24
2	192.168.2.0/24
3	192.168.2.0/28
4	
5	

TCAM in operation (example 2)

- Given the input in the figure, what the TCAM would provide? *It depends on its configuration 😊*
- TCAMs report only one match. It can be the highest or the lowest address
- It is important to order the entries in the correct way

address	entry
0	192.168.3.0/24
1	192.168.1.0/24
2	192.168.2.0/24
3	192.168.2.0/28
4	
5	

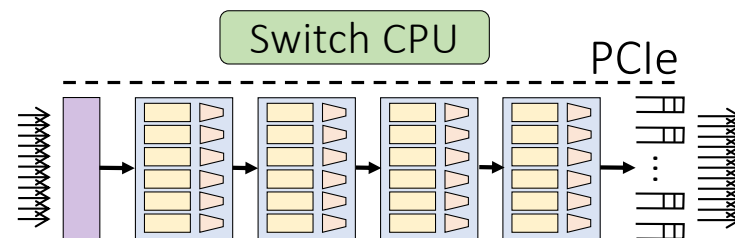
192.168.2.1 →

TCAM in operation (example 2)

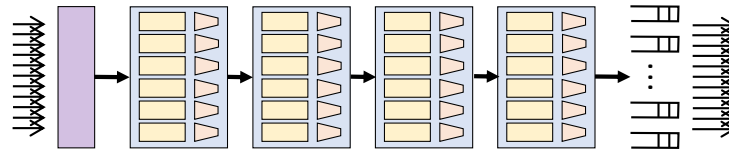
- Given the input in the figure, what the TCAM would provide? *It depends on its configuration 😊*
- TCAMs report only one match. It can be the highest or the lowest address
- It is important to order the entries in the correct way
- This requires additional logic!

address	entry
0	192.168.3.0/24
1	192.168.1.0/24
2	192.168.2.0/24
3	192.168.2.0/28
4	
5	

192.168.2.1 →



TCAM is not the only available resource



- Each stage has also some SRAM memory available
- SRAM can be used to store data and retrieve it later (stateful operations)
- The challenge: perform storing and lookup using *not much* memory!

Implementing a set

- We want to store in memory some data and being able to retrieve it

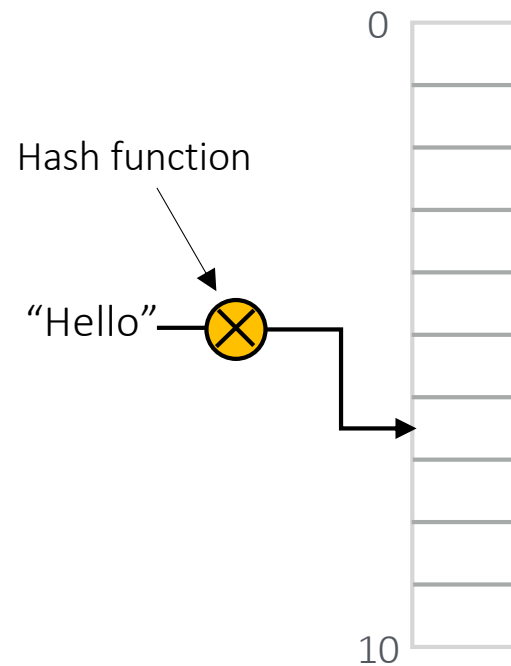
	strategy #1	strategy #2
output	deterministic	probabilistic
number of required operations	probabilistic	deterministic

Implementing a set

- We want to store in memory some data and being able to retrieve it

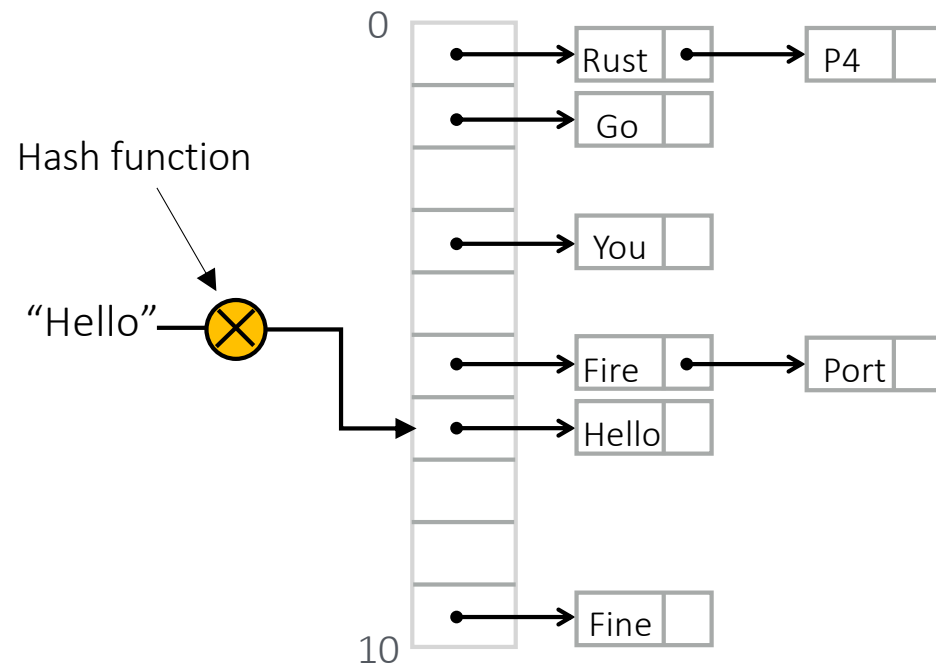
	strategy #1	strategy #2
output	deterministic	probabilistic
number of required operations	probabilistic	deterministic

Intuitive implementation of a set Separate-Chaining



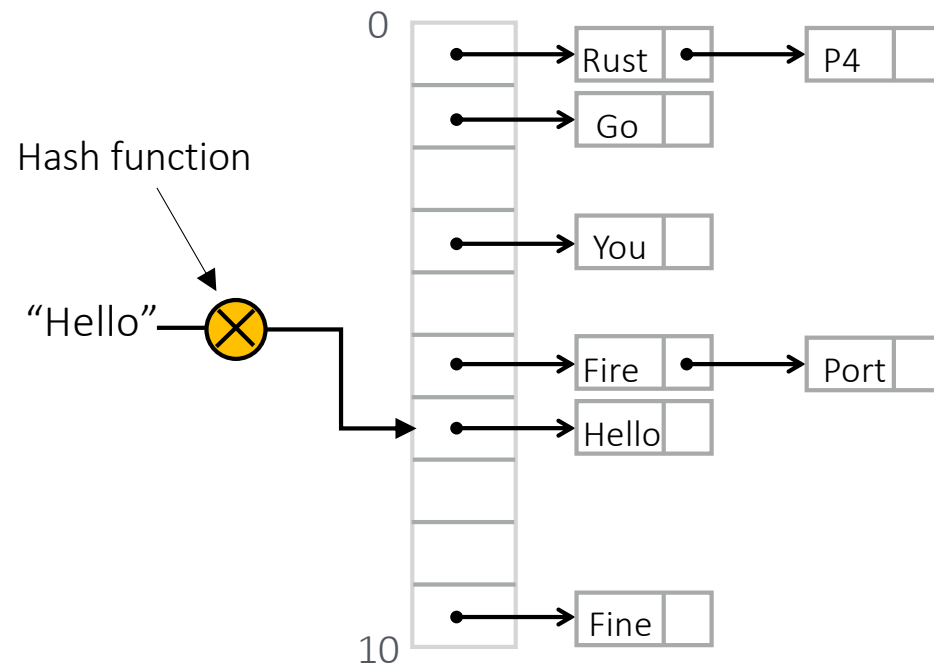
- A hash function is any function that can be used to map data of arbitrary size to fixed-size values
- It basically converts a given numeric or alphanumeric key to a small practical integer value
- The mapped integer value is used as an index in the hash table

Intuitive implementation of a set Separate-Chaining



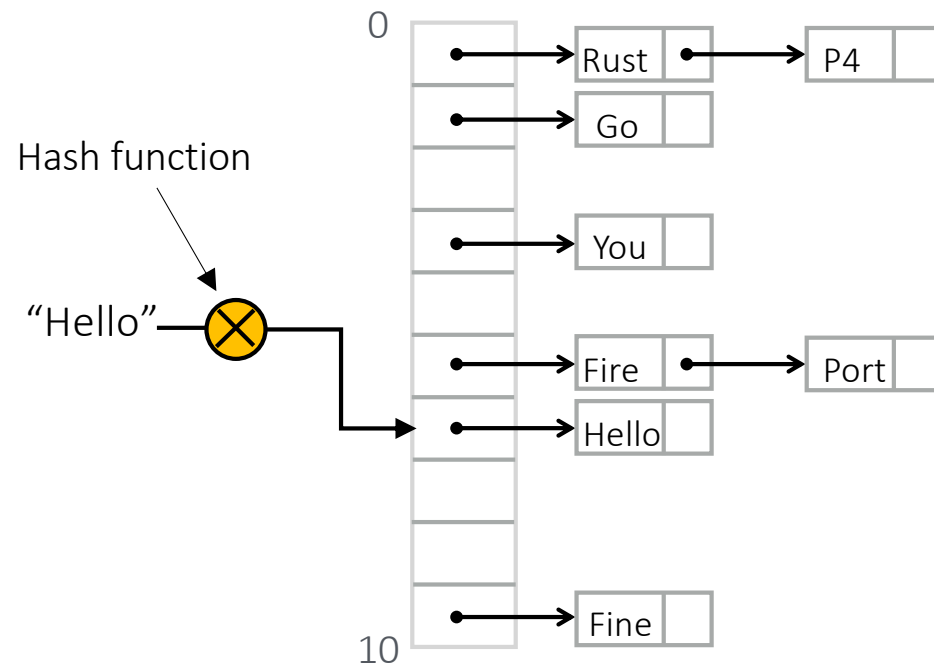
- If multiple values results in the same hash index, then we can chain the data and create a list

Intuitive implementation of a set Separate-Chaining



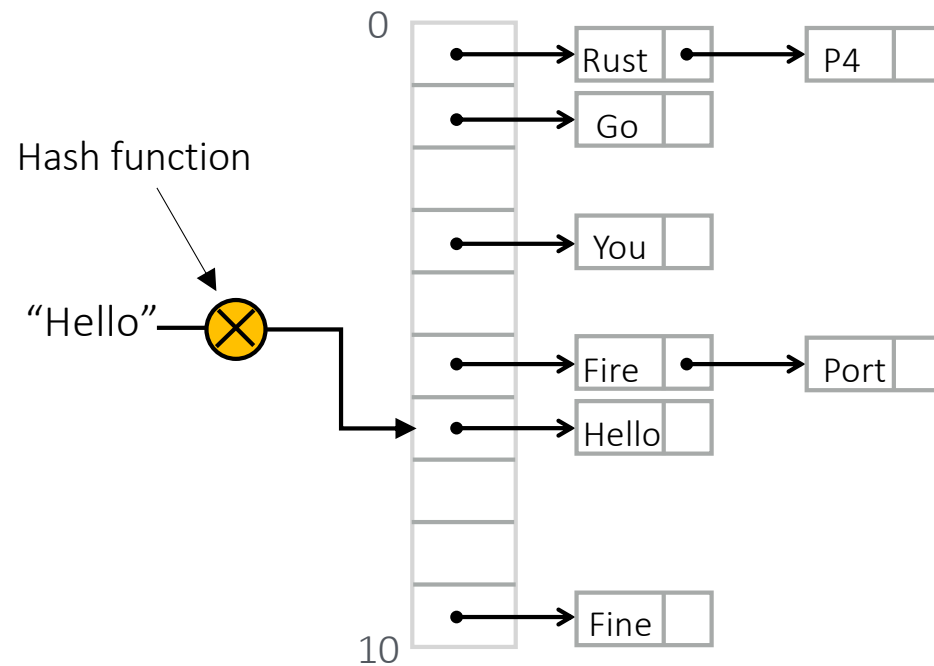
- If multiple values results in the same hash index, then we can chain the data and create a list
- If we have N elements and M cells then:
 - Average list size?

Intuitive implementation of a set Separate-Chaining



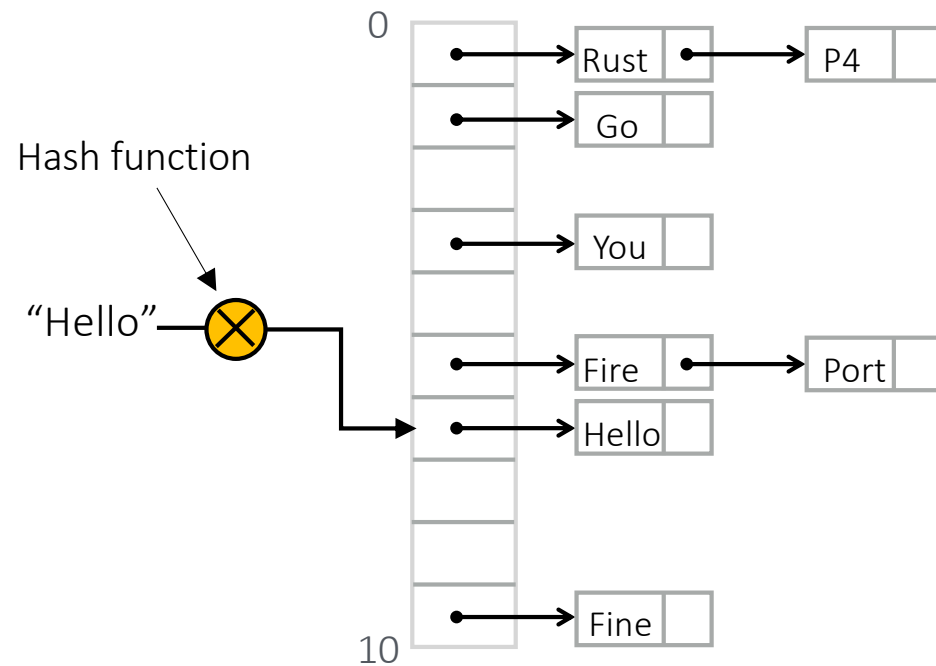
- If multiple values results in the same hash index, then we can chain the data and create a list
- If we have N elements and M cells then:
 - Average list size? N/M
 - Worst case list size?

Intuitive implementation of a set Separate-Chaining



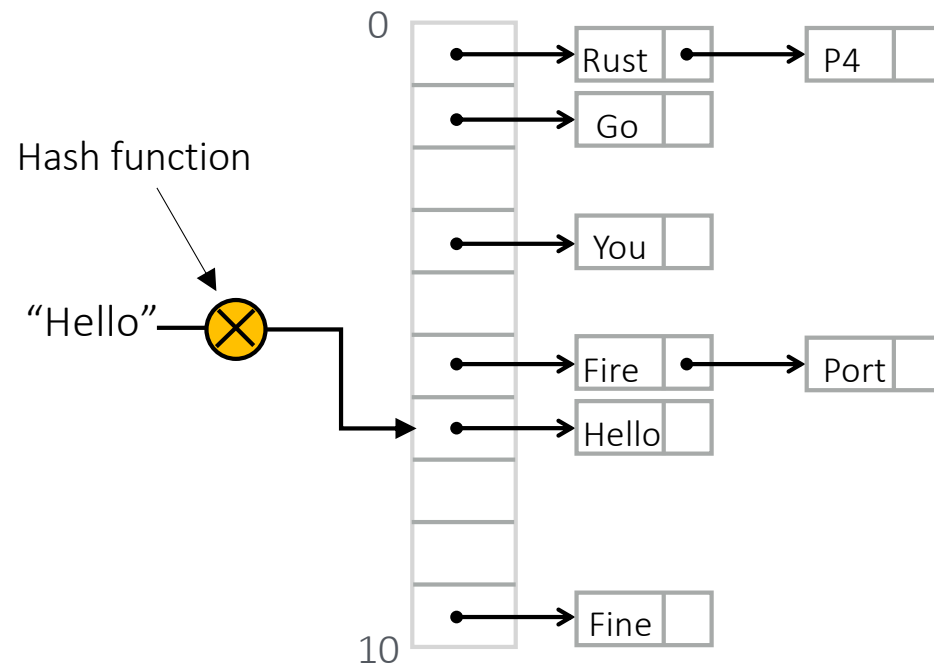
- If multiple values results in the same hash index, then we can chain the data and create a list
- If we have N elements and M cells then:
 - Average list size? N/M
 - Worst case list size? N

Intuitive implementation of a set Separate-Chaining



- The first insertion does not collide
- The second insertion does not collide with probability: $1 - \frac{1}{M}$
- The third insertion does not collide with probability: $1 - \frac{2}{M}$
- The N-th insertion does not collide with probability: $1 - \frac{N-1}{M}$

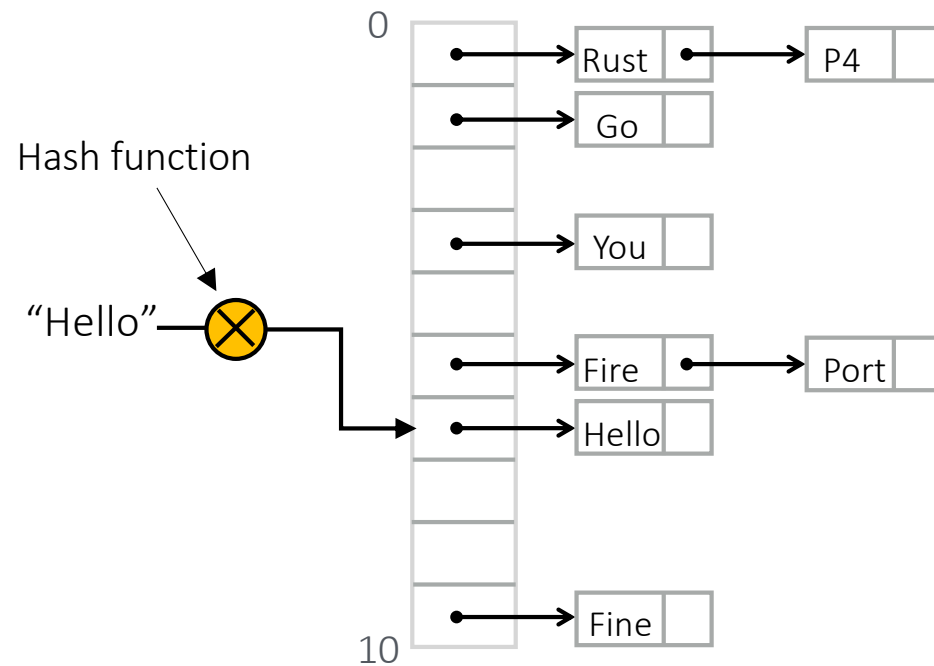
Intuitive implementation of a set Separate-Chaining



The probability that none of the insertions collide is:

$$P(N, M) = \left(1 - \frac{1}{M}\right) \left(1 - \frac{2}{M}\right) \dots \left(1 - \frac{N-1}{M}\right) = \prod_{i=1}^{N-1} \left(1 - \frac{i}{M}\right)$$

Intuitive implementation of a set Separate-Chaining

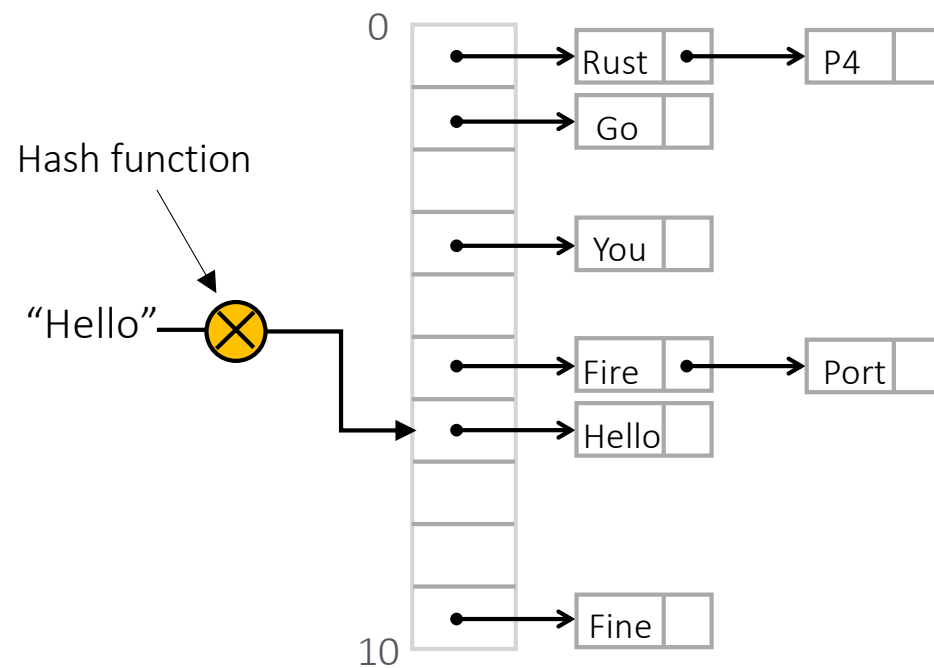


The probability that none of the insertions collide is:

$$P(N,M) = \left(1 - \frac{1}{M}\right)\left(1 - \frac{2}{M}\right)\dots\left(1 - \frac{N-1}{M}\right) = \prod_{i=1}^{N-1} \left(1 - \frac{i}{M}\right)$$

With $N=100$ and $M = 10000$ the probability of having 1 or more collisions is already around 40%!!!

Intuitive implementation of a set Separate-Chaining



PRO: accurate

CON: efficient if there is a low number of elements to be stored

Implementing a set

- We want to store in memory some data and being able to retrieve it

	strategy #1	strategy #2
output	deterministic	probabilistic
number of required operations	probabilistic	deterministic

Implementing a set

- We want to store in memory some data and being able to retrieve it

	strategy #1	strategy #2
output	deterministic	probabilistic
number of required operations	probabilistic	deterministic

probabilistic data structures

Implementing a set

- We want to store in memory some data and being able to retrieve it

	strategy #1	strategy #2
output	deterministic	probabilistic
number of required operations	probabilistic	deterministic

probabilistic data structures

We like this aspect (recall the stages in the PISA architecture)

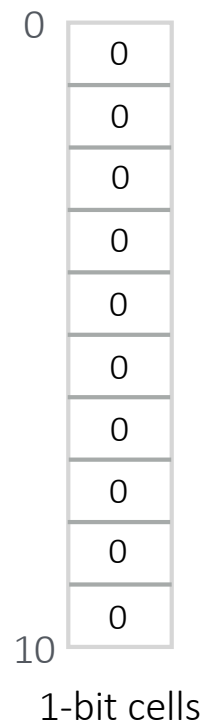
Implementing a set

- We want to store in memory some data and being able to retrieve it

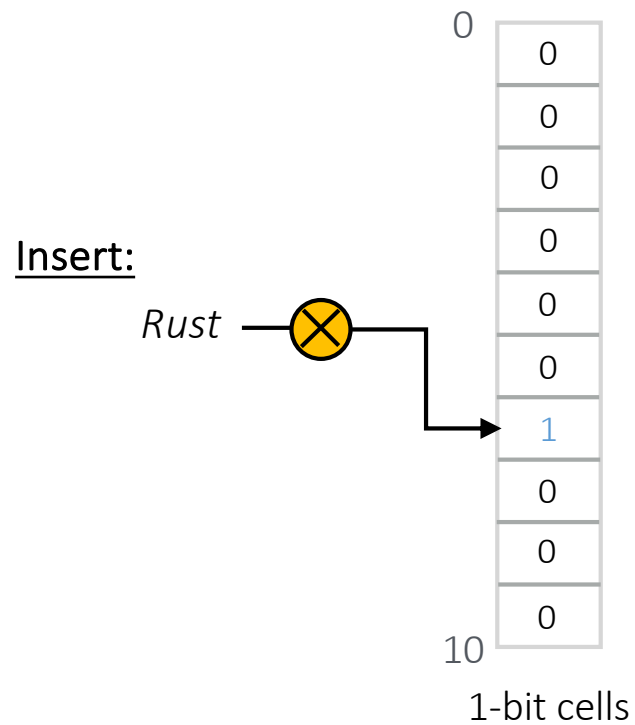
	strategy #1	strategy #2	
output	deterministic	probabilistic	We like this aspect a bit less 😊 but we can work around it
number of required operations	probabilistic	deterministic	

probabilistic data structures

Insertions and membership queries

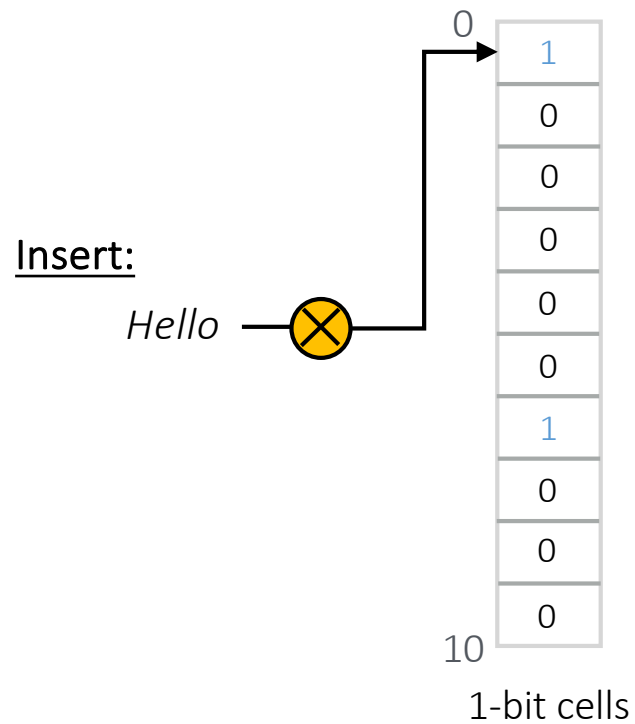


Insertions and membership queries



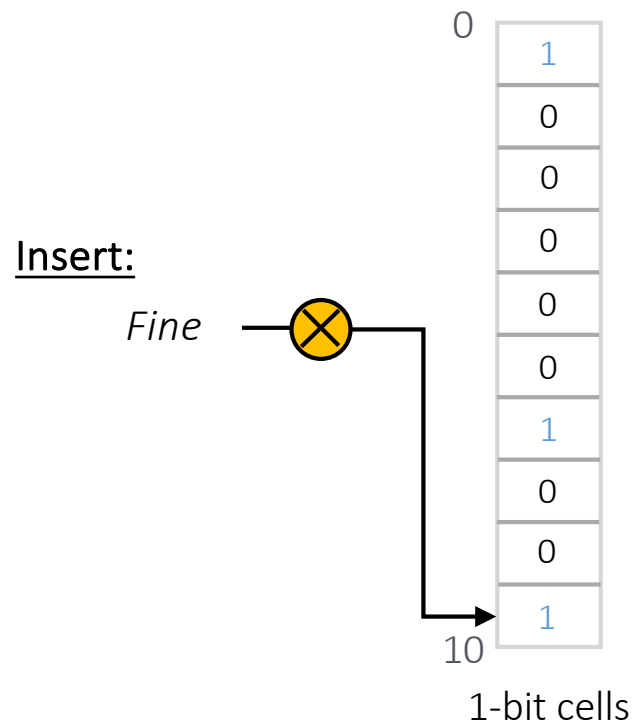
- So far we inserted {Rust}

Insertions and membership queries



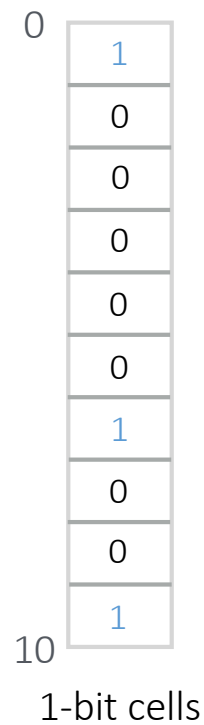
- So far we inserted {Rust, Hello}

Insertions and membership queries



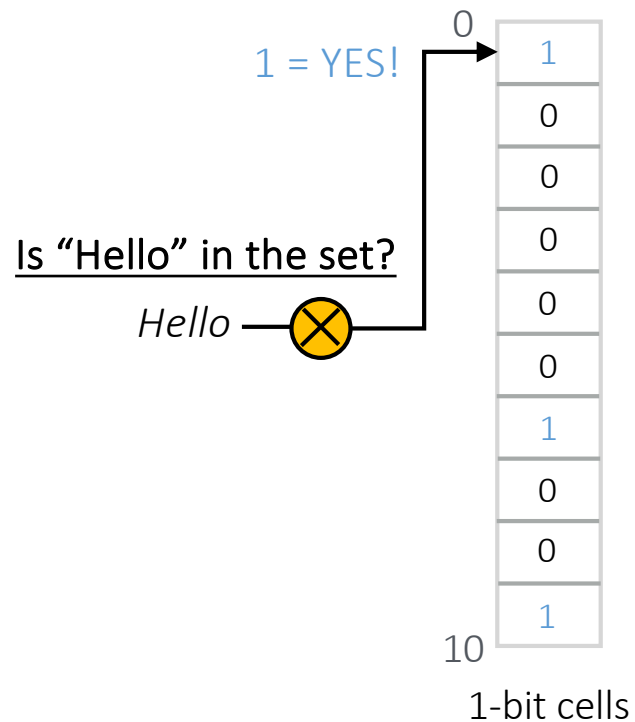
- So far we inserted {Rust, Hello, Fine}

Insertions and membership queries



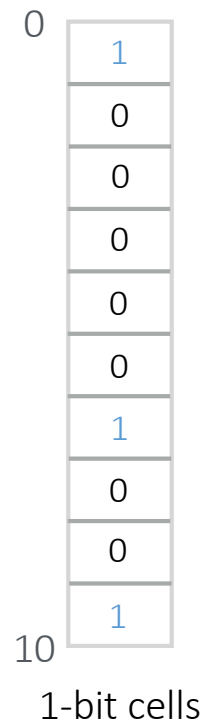
- So far we inserted {Rust, Hello, Fine}
- We do only have the 10-bit array. How do we know if Hello is inserted?

Insertions and membership queries



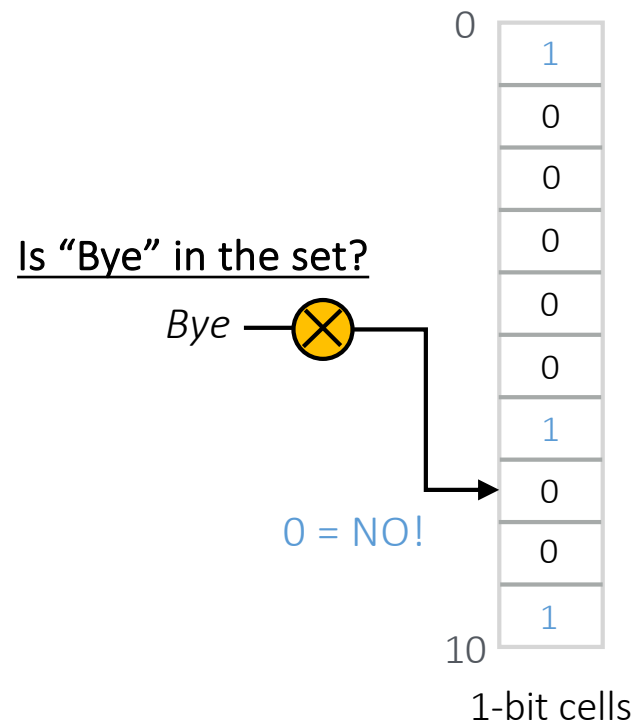
- So far we inserted {Rust, Hello, Fine}
- We do only have the 10-bit array. How do we know if Hello is inserted?
- We lookup up at the h(hello) address and check if there is 1 or 0!

Insertions and membership queries



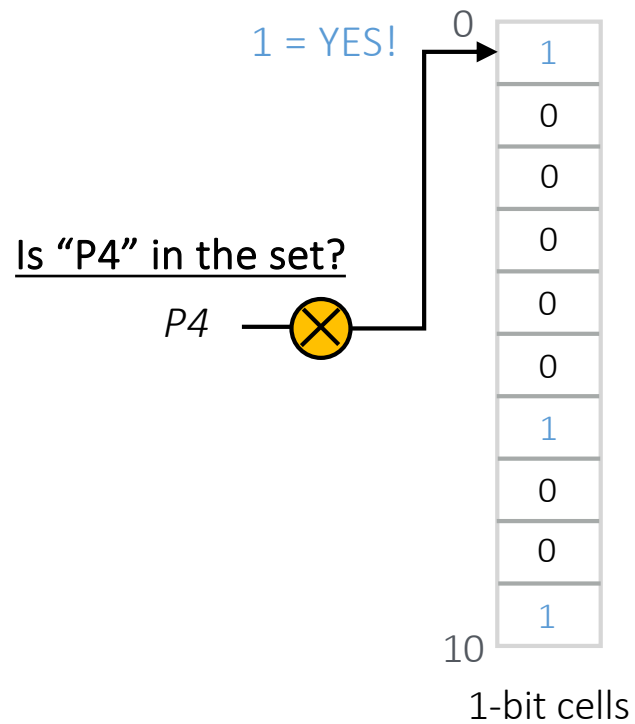
- So far we inserted {Rust, Hello, Fine}
- Is Bye in the set?

Insertions and membership queries



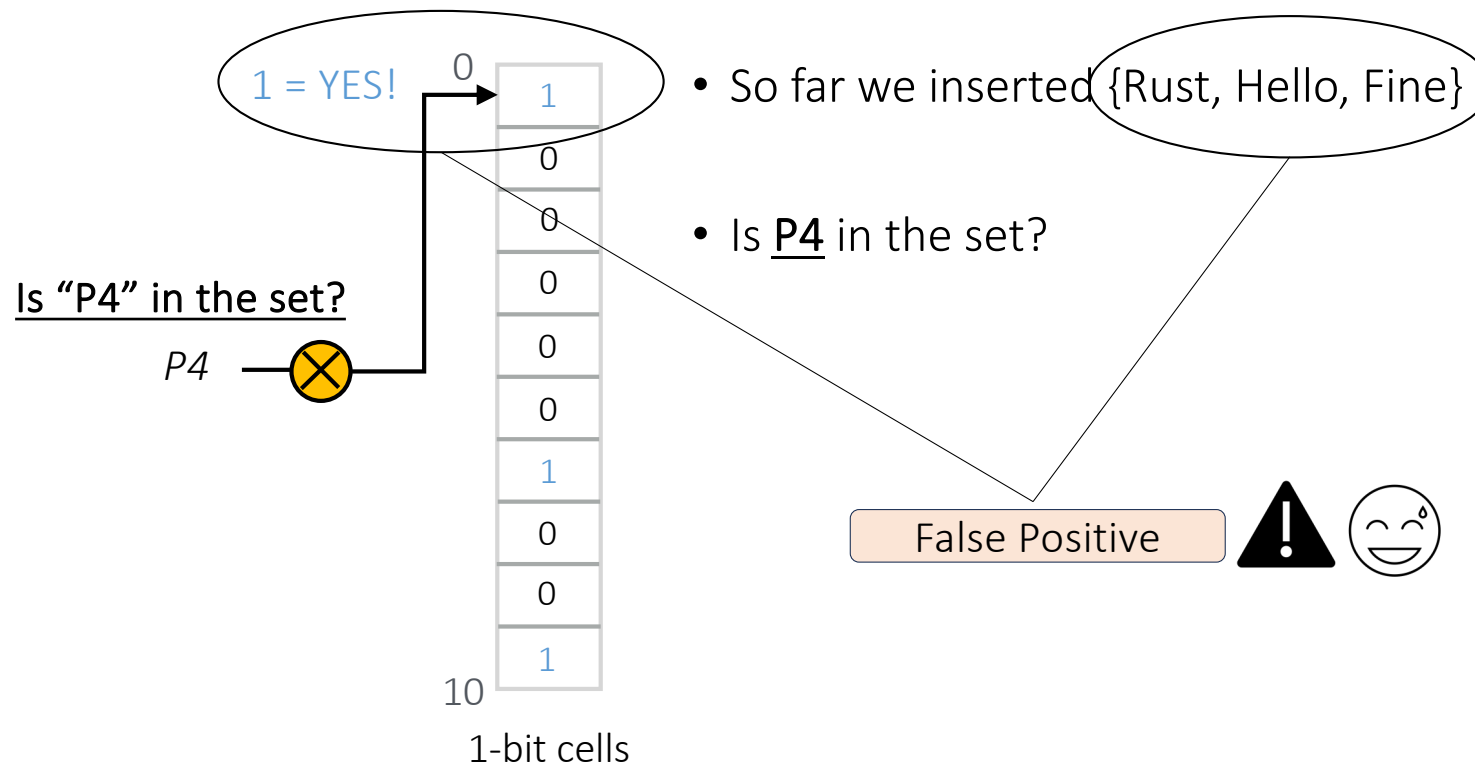
- So far we inserted {Rust, Hello, Fine}
- Is Bye in the set?

Insertions and membership queries

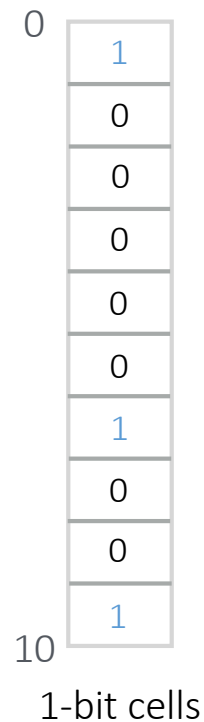


- So far we inserted {Rust, Hello, Fine}
- Is P4 in the set?

Insertions and membership queries



Insertions and membership queries



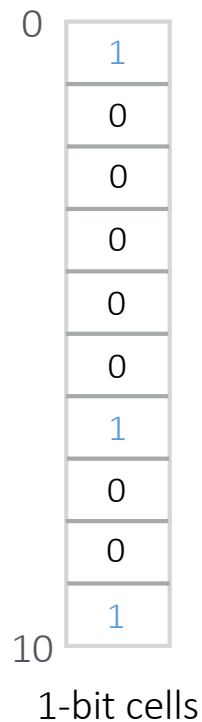
Let's assume we have N elements and M cells

- Probability of an element to be mapped to a particular cell: $\frac{1}{M}$
- Probability of an element *not to* be mapped to a particular cell: $1 - \frac{1}{M}$
- Probability of a cell to be zero: $(1 - \frac{1}{M})^N$
- False Positive Rate: $1 - (1 - \frac{1}{M})^N$
- False Negative Rate: 0

Insertions and membership queries

If $N=1000$ and $M=10000$ then $FPR=9.5\%$

If $N=1000$ and $M=100000$ then $FPR=1\%$



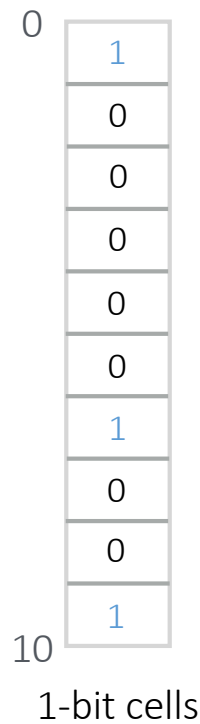
Let's assume we have N elements and M cells

- Probability of an element to be mapped to a particular cell: $\frac{1}{M}$
- Probability of an element *not to* be mapped to a particular cell: $1 - \frac{1}{M}$
- Probability of a cell to be zero: $(1 - \frac{1}{M})^N$
- False Positive Rate: $1 - (1 - \frac{1}{M})^N$
- False Negative Rate: 0

Insertions and membership queries

If $N=1000$ and $M=10000$ then $FPR=9.5\%$

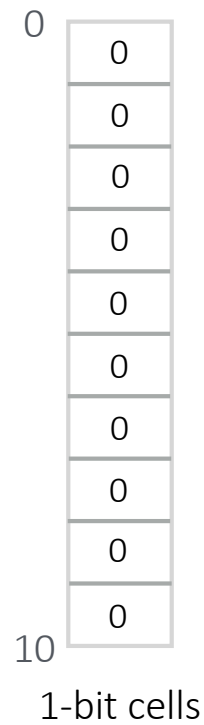
If $N=1000$ and $M=100000$ then $FPR=1\%$



PRO: simple and deterministic number of operations (both for insertion and queries)

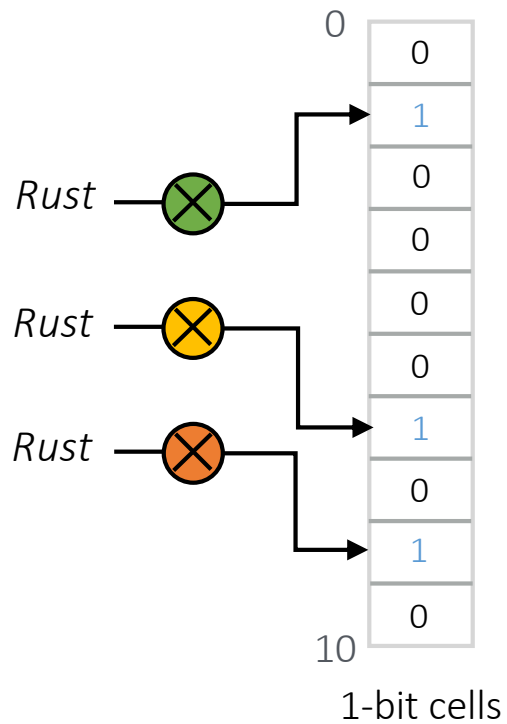
CON: we need roughly 100x more cells than the number of elements we need to store for a 1% false positive rate ☹️

Bloom Filters: a more memory-efficient approach



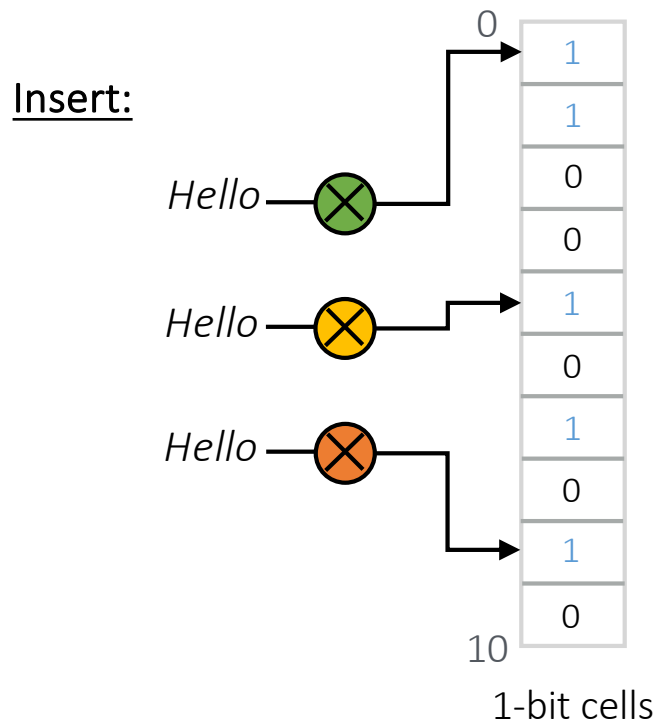
Bloom Filters: a more memory-efficient approach

Insert:



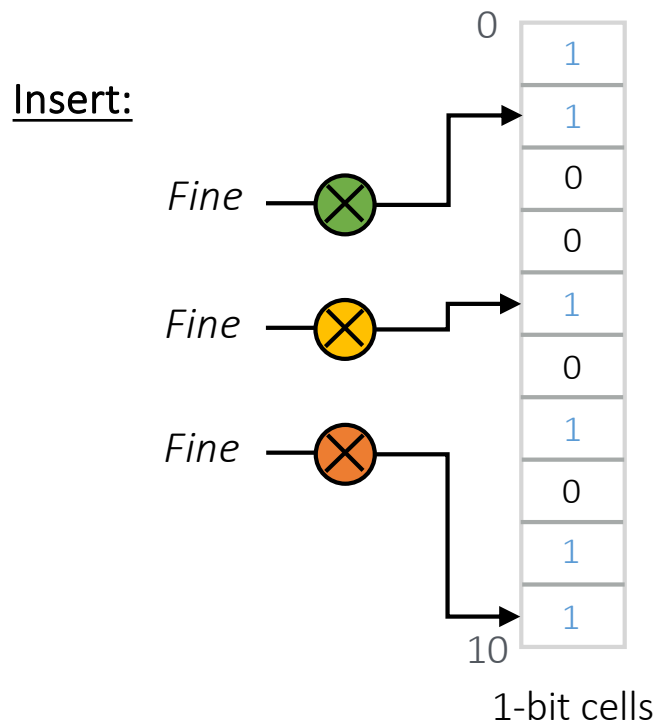
- So far we inserted {Rust}

Bloom Filters: a more memory-efficient approach



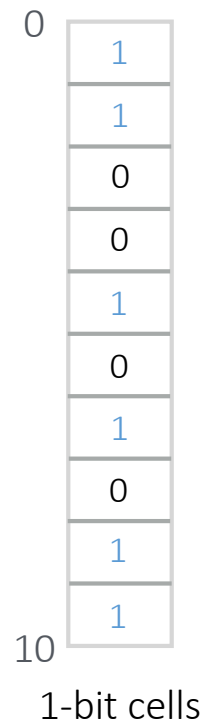
- So far we inserted {Rust, Hello}
- To be noted that one of the cells (number 9) was set already to 1 by *Rust*

Bloom Filters: a more memory-efficient approach



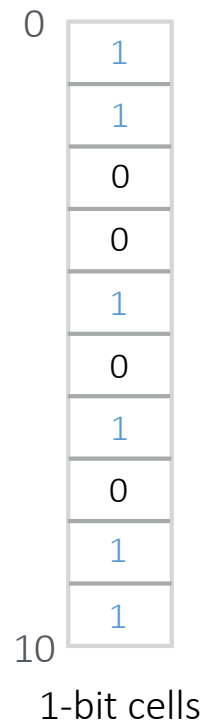
- So far we inserted {Rust, Hello, Fine}
- To be noted that two of the cells (cell number 1 and 4) were set already to 1 by the previous elements

Bloom Filters: a more memory-efficient approach



- So far we inserted {Rust, Hello, Fine}
- How to know if an element is present or not in the set?

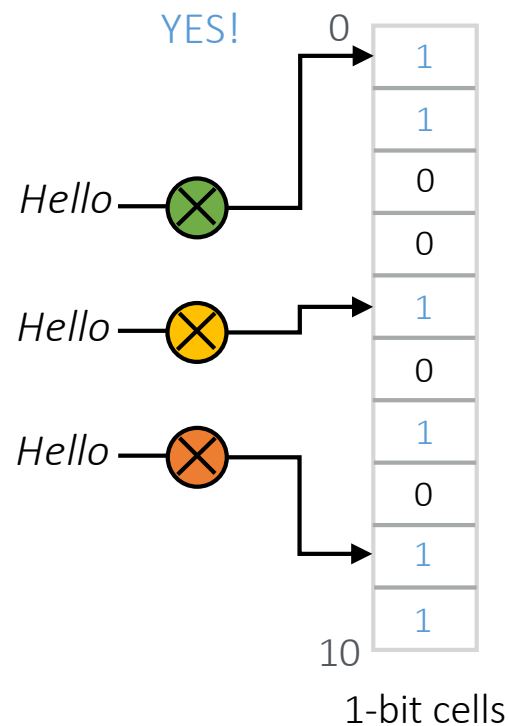
Bloom Filters: a more memory-efficient approach



- So far we inserted {Rust, Hello, Fine}
- How to know if an element is present or not in the set?
 - An element is considered in the set if all the hash values map to a cell with 1
 - An element is not in the set if at least one hash value maps to a cell with 0

Bloom Filters: a more memory-efficient approach

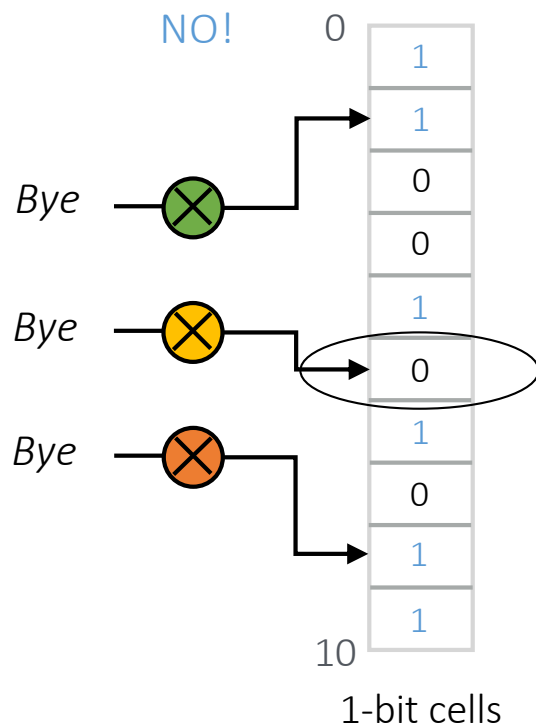
Is "Hello" in the set?



- So far we inserted {Rust, Hello, Fine}
- How to know if an element is present or not in the set?
 - An element is considered in the set if all the hash values map to a cell with 1
 - An element is not in the set if at least one hash value maps to a cell with 0

Bloom Filters: a more memory-efficient approach

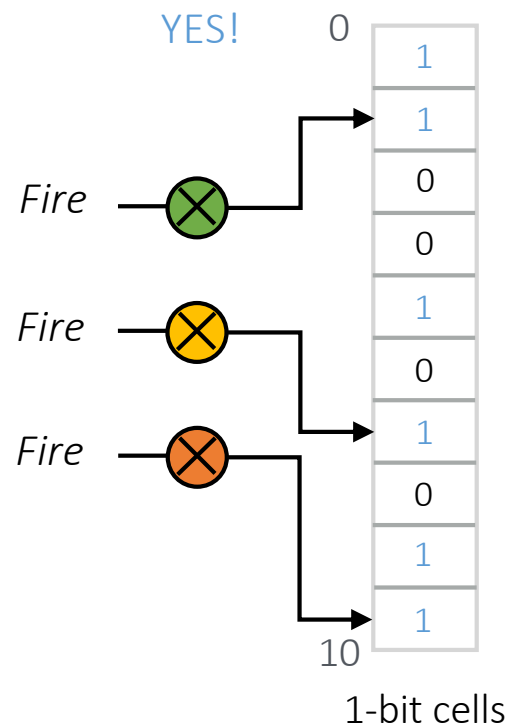
Is “Bye” in the set?



- So far we inserted {Rust, Hello, Fine}
- How to know if an element is present or not in the set?
 - An element is considered in the set if **all** the hash values map to a cell with 1
 - An element is not in the set if at least one hash value maps to a cell with 0

Bloom Filters: a more memory-efficient approach

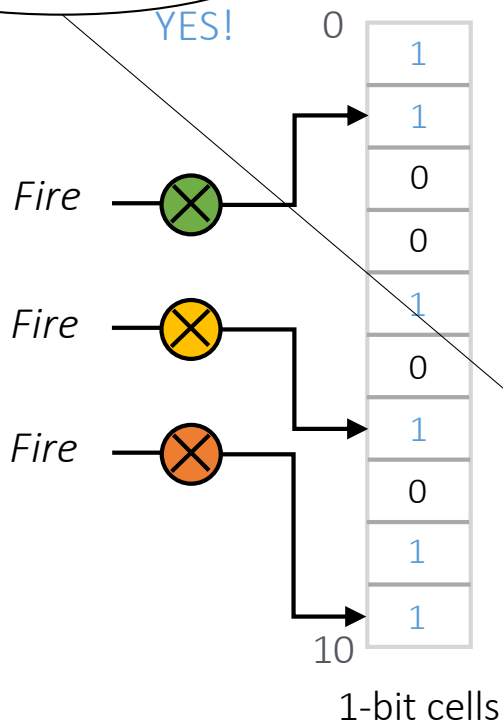
Is "Fire" in the set?



- So far we inserted {Rust, Hello, Fine}
- How to know if an element is present or not in the set?
 - An element is considered in the set if all the hash values map to a cell with 1
 - An element is not in the set if at least one hash value maps to a cell with 0

Bloom Filters: a more memory-efficient approach

Is "Fire" in the set?



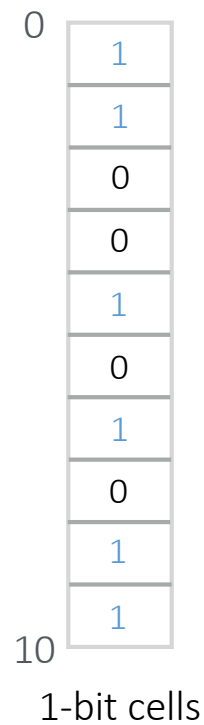
• So far we inserted {Rust, Hello, Fine}

- How to know if an element is present or not in the set?
 - An element is considered in the set if all the hash values map to a cell with 1
 - An element is not in the set if at least one hash value maps to a cell with 0

False Positive



Bloom Filters: a more memory-efficient approach



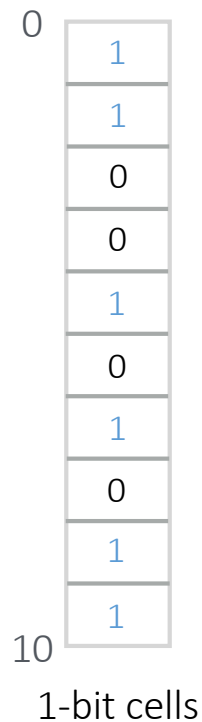
Let's assume we have N elements M cells and K hash functions

- Probability one hash function return the index of a particular cell: $\frac{1}{M}$
- Probability one hash function does not return the index of a particular cell: $1 - \frac{1}{M}$
- Probability of a cell to be zero: $(1 - \frac{1}{M})^{KN}$
- False Positive Rate: $(1 - (1 - \frac{1}{M})^{KN})^K$
- False Negative Rate: 0

Bloom Filters: a more memory-efficient approach

If $N=1000$,
 $M=10000$ and
we have 7 hash
functions then
 $FPR=0.82\%$

If $N=1000$,
 $M=100000$ and
we have 7 hash
functions then
 $FPR \text{ approx. } 0\%$



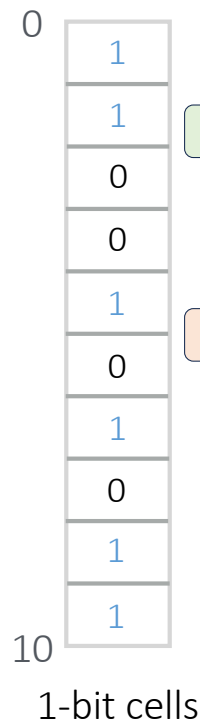
Let's assume we have N elements M cells and K hash functions

- Probability one hash function return the index of a particular cell: $\frac{1}{M}$
- Probability one hash function does not return the index of a particular cell: $1 - \frac{1}{M}$
- Probability of a cell to be zero: $(1 - \frac{1}{M})^{KN}$
- False Positive Rate: $(1 - (1 - \frac{1}{M})^{KN})^K$
- False Negative Rate: 0

Bloom Filters: a more memory-efficient approach

If $N=1000$,
 $M=10000$ and
we have 7 hash
functions then
 $FPR=0.82\%$

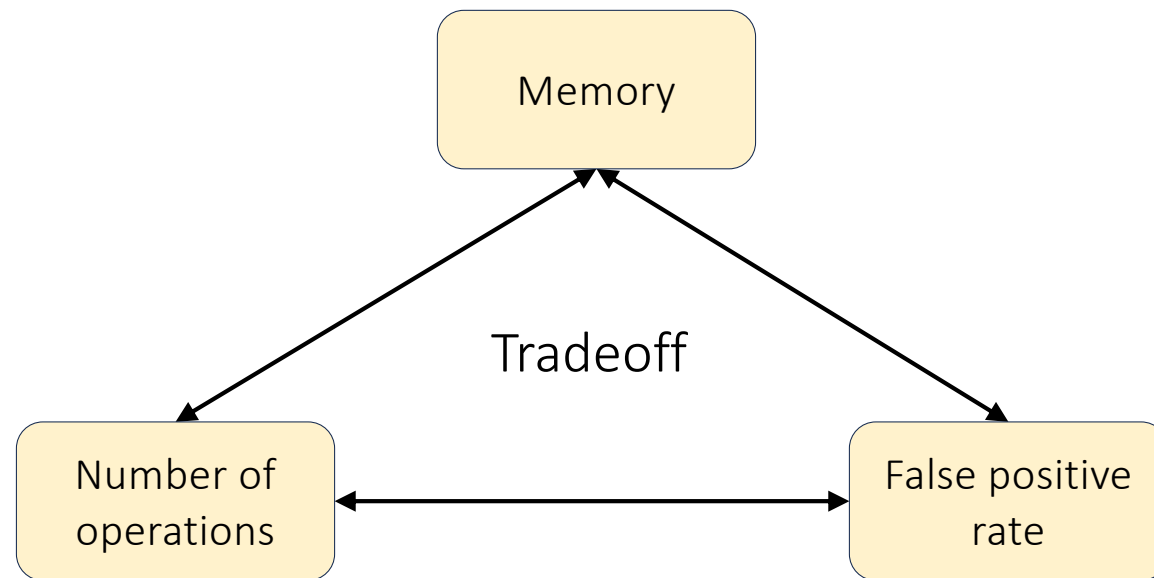
If $N=1000$,
 $M=100000$ and
we have 7 hash
functions then
 $FPR \text{ approx. } 0\%$



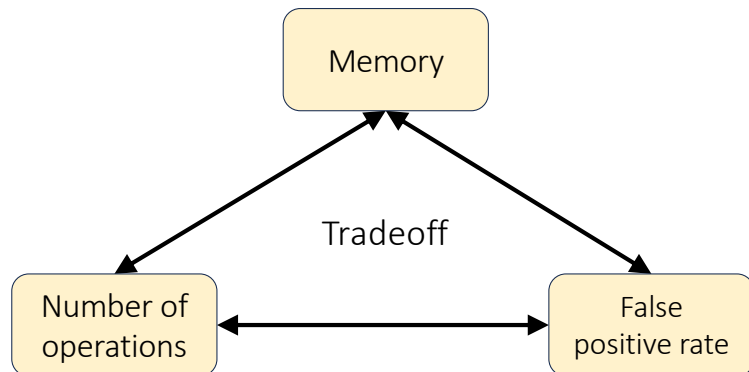
PRO: consumes roughly 10x less memory than the simple approach

CON: requires slightly more operations than the simple approach (7 hashes instead of 1)

Dimension your Bloom Filter



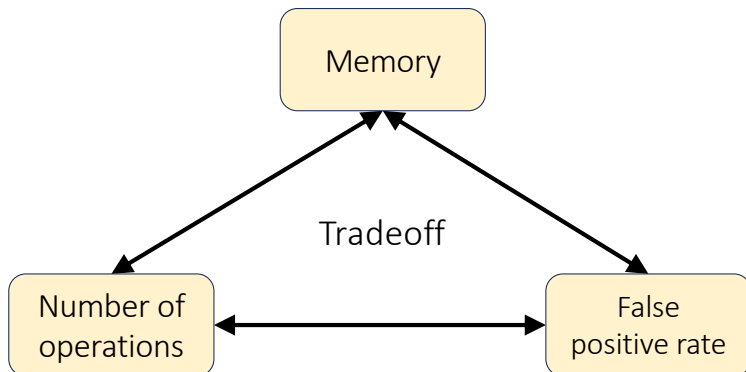
Dimension your Bloom Filter



Let's assume we have N elements M cells and K hash functions

$$\text{False Positive Rate: } \left(1 - \left(1 - \frac{1}{M}\right)^{KN}\right)^K \cong (1 - e^{-KN/M})^K$$

Dimension your Bloom Filter



Let's assume we have N elements M cells and K hash functions

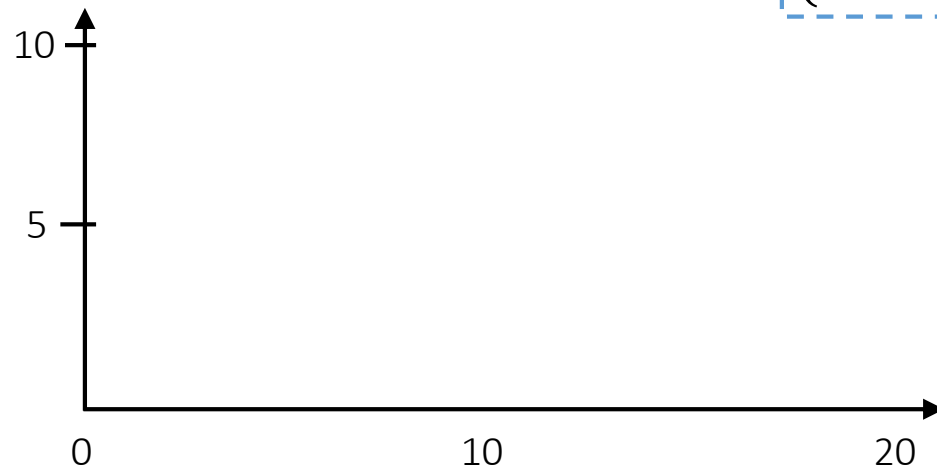
$$\text{False Positive Rate: } \left(1 - \left(1 - \frac{1}{M}\right)^{KN}\right)^K \cong \left(1 - e^{-KN/M}\right)^K$$

asymptotic approximation

Dimension your Bloom Filter

Let's assume we have N elements M cells and K hash functions

False Positive
Rate (%)



K (number of hash functions)

$$\text{False Positive Rate: } \left(1 - \left(1 - \frac{1}{M}\right)^{KN}\right)^K \cong \left(1 - e^{-KN/M}\right)^K$$

asymptotic approximation

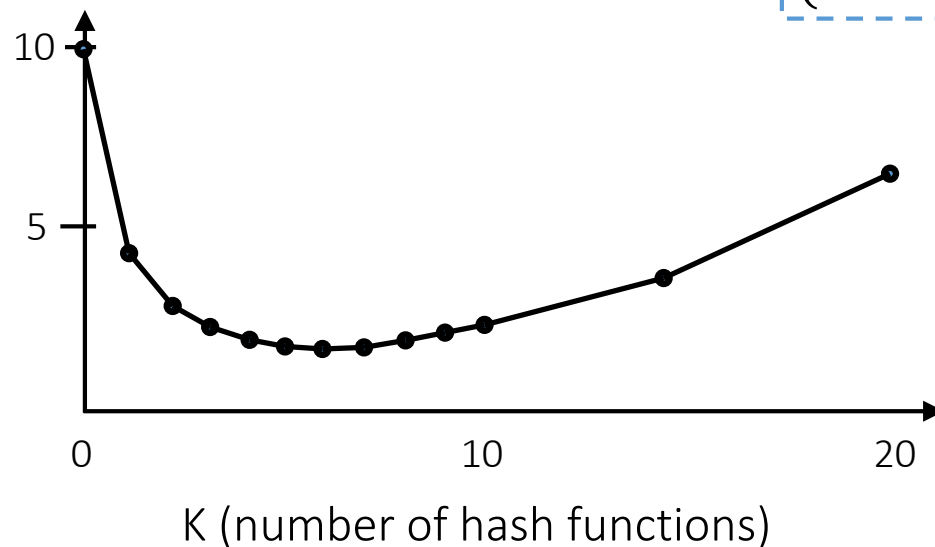
Dimension your Bloom Filter

Let's assume we have N elements M cells and K hash functions

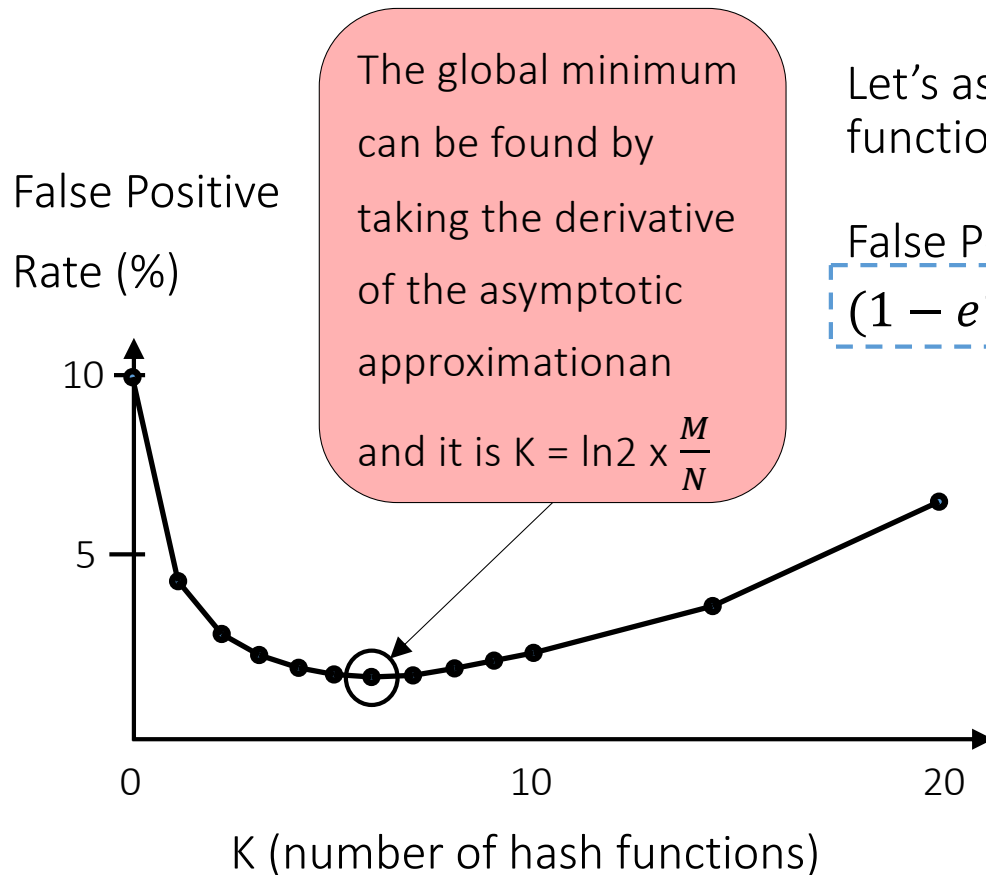
False Positive
Rate (%)

$$\text{False Positive Rate: } \left(1 - \left(1 - \frac{1}{M}\right)^{KN}\right)^K \cong \left(1 - e^{-KN/M}\right)^K$$

asymptotic approximation



Dimension your Bloom Filter



Let's assume we have N elements M cells and K hash functions

False Positive Rate: $(1 - (1 - \frac{1}{M})^{KN})^K \cong$
 $(1 - e^{-KN/M})^K$
 asymptotic approximation

Implementing a set

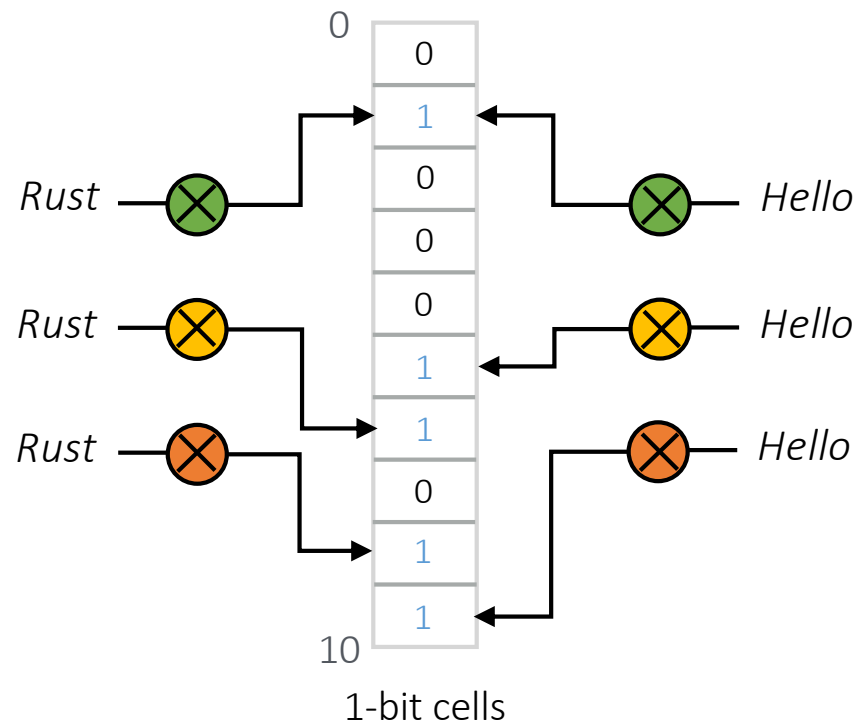
- We want to store in memory some data and being able to retrieve it

	strategy #1	strategy #2
output	deterministic	probabilistic
number of required operations	probabilistic	deterministic

Bloom Filters

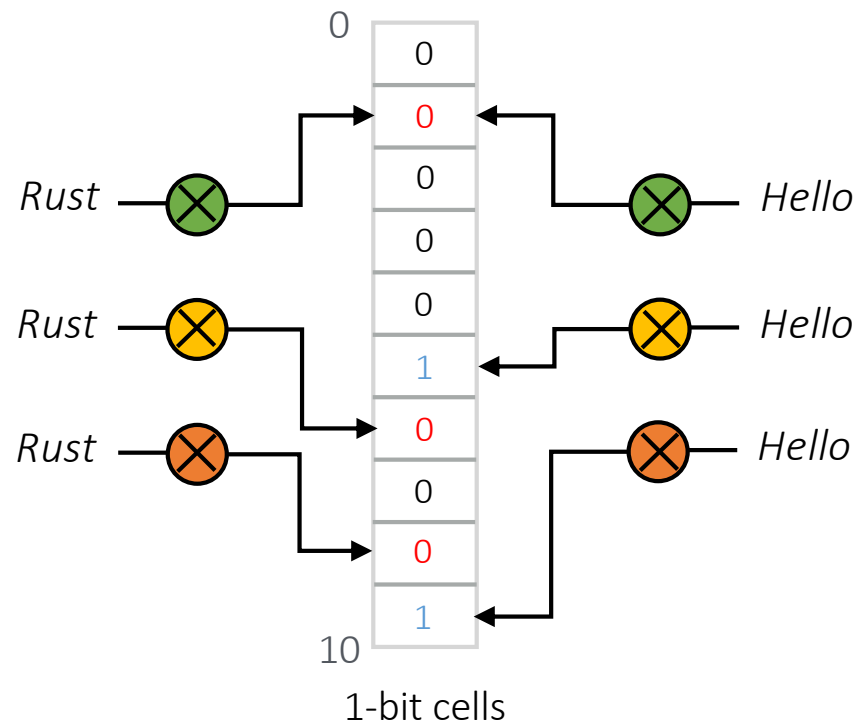
What if we want to delete an entry stored in a Bloom Filter?

What if we want to delete an entry stored in a Bloom Filter?



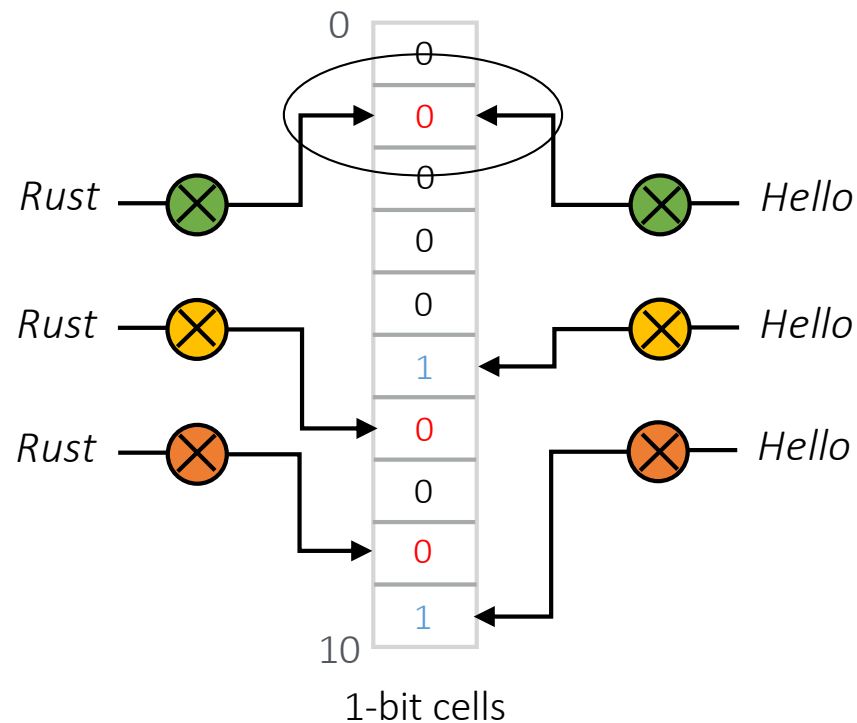
- Let's assume we add to the set {Rust, Hello}

What if we want to delete an entry stored in a Bloom Filter?



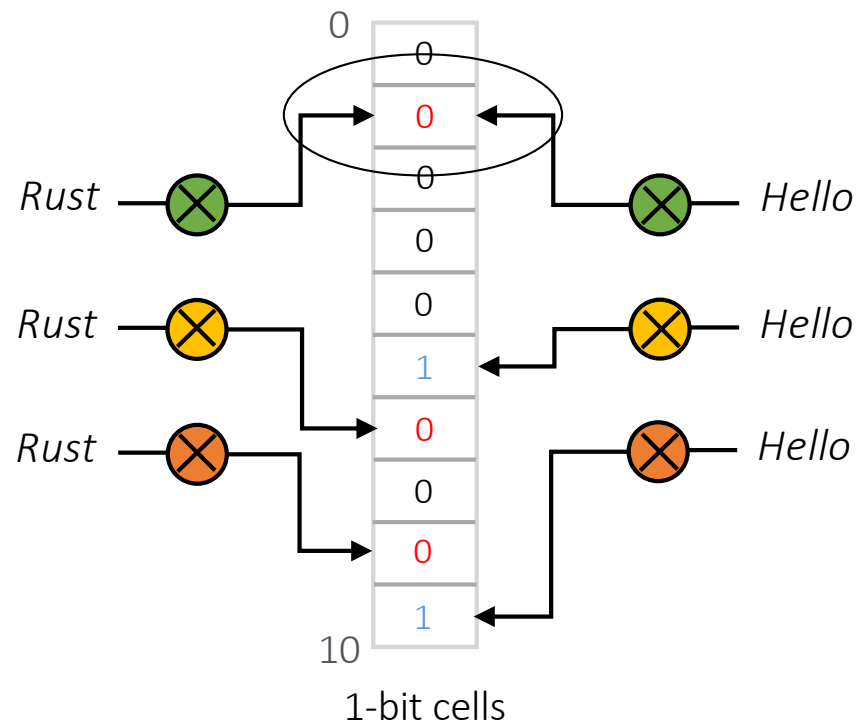
- Let's assume we add to the set {Rust, Hello}
- What happens if we want to delete **Rust**?

What if we want to delete an entry stored in a Bloom Filter?



- Let's assume we add to the set {Rust, Hello}
- What happens if we want to delete **Rust**?
- Deleting **Rust** forces us to delete also **Hello**

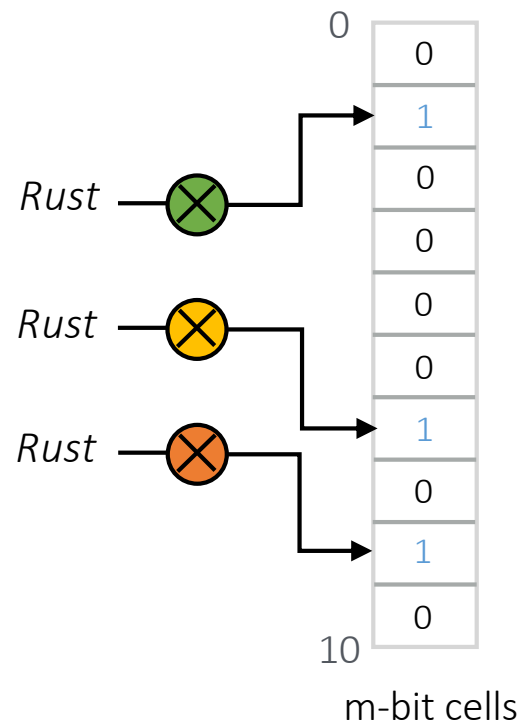
What if we want to delete an entry stored in a Bloom Filter?



- Let's assume we add to the set {Rust, Hello}
- What happens if we want to delete **Rust**?
- Deleting **Rust** forces us to delete also **Hello**
- How to deal with this?

Counting Bloom Filters

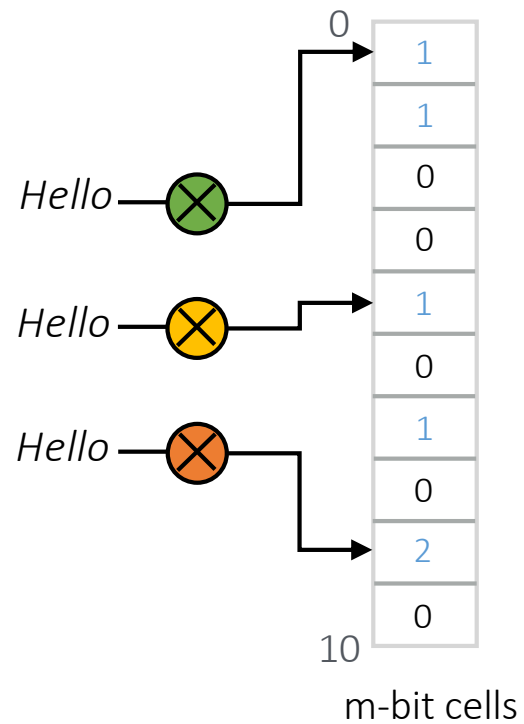
Insert:



- So far we inserted {Rust}

Counting Bloom Filters

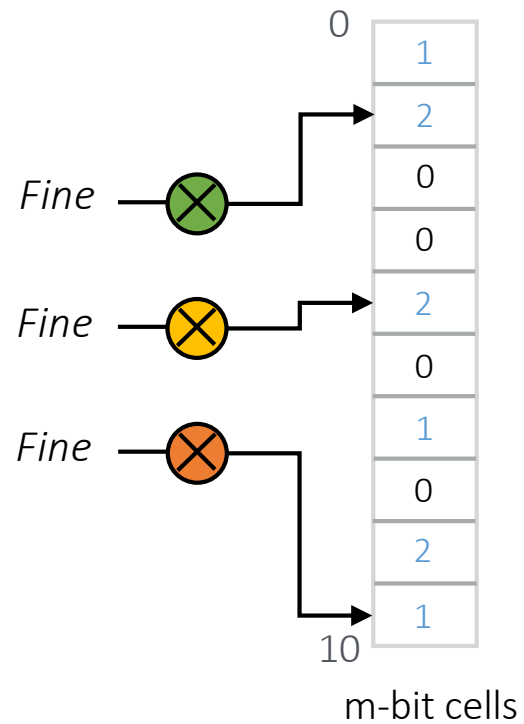
Insert:



- So far we inserted {Rust, Hello}
- To add an element, increment the corresponding counter

Counting Bloom Filters

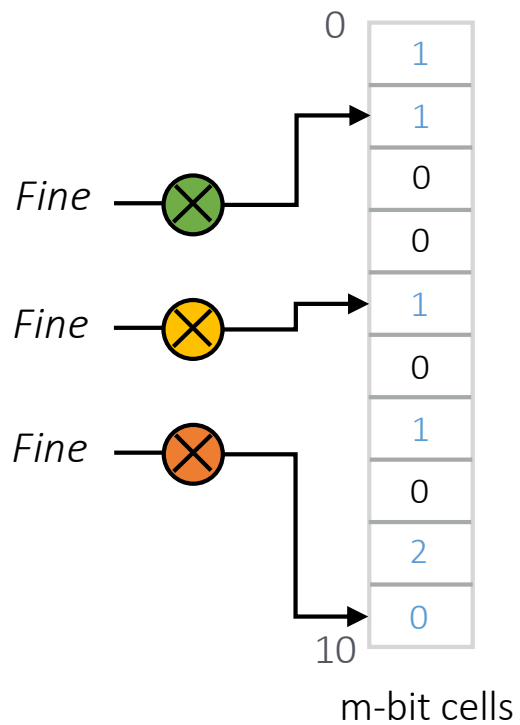
Insert:



- So far we inserted {Rust, Hello, Fine}
- To add an element, increment the corresponding counter

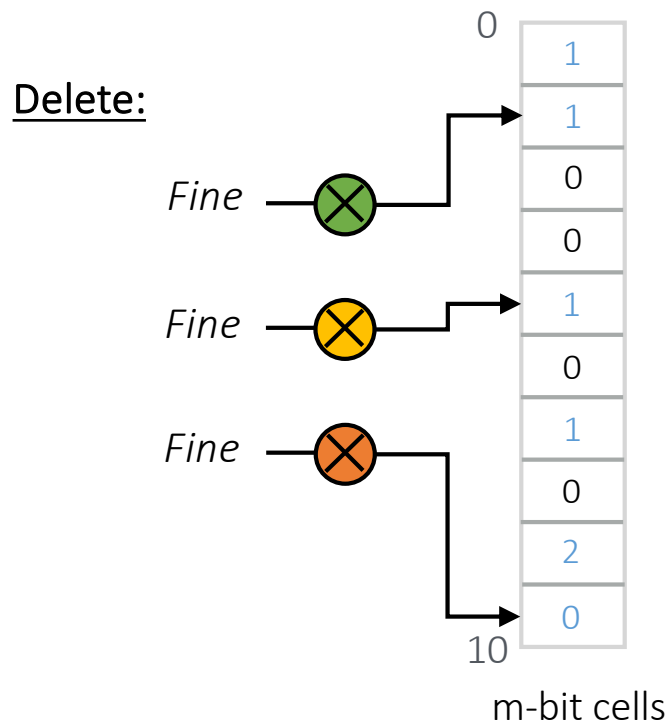
Counting Bloom Filters

Delete:



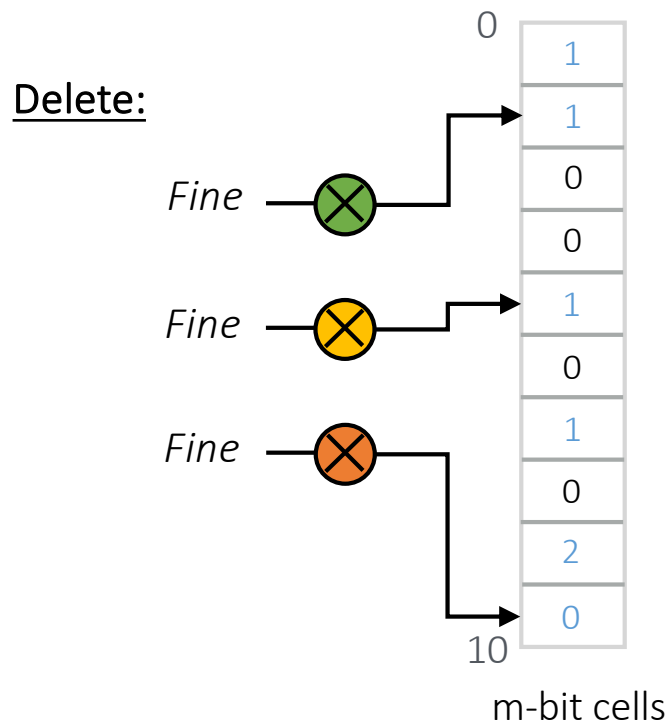
- So far we inserted {Rust, Hello}
- To add an element, increment the corresponding counter
- To delete an element, decrement the corresponding counter

Counting Bloom Filters



- Counting Bloom Filters do handle deletions at the price of using more memory
- Counters must be large enough to avoid overflow. (Why?)

Counting Bloom Filters



- Counting Bloom Filters do handle deletions at the price of using more memory
- Counters must be large enough to avoid overflow. (Why?)
- If a counter overflows, the filter may yield false negatives!

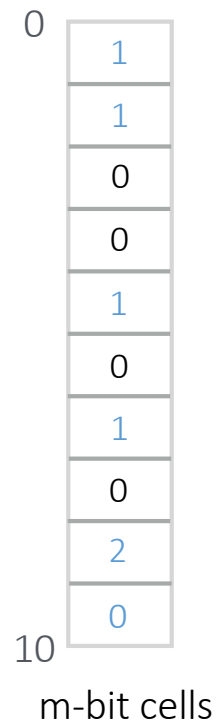
Implementing a set

- We want to store in memory some data and being able to retrieve it

	strategy #1	strategy #2
output	deterministic	probabilistic
number of required operations	probabilistic	deterministic

Counting Bloom Filters

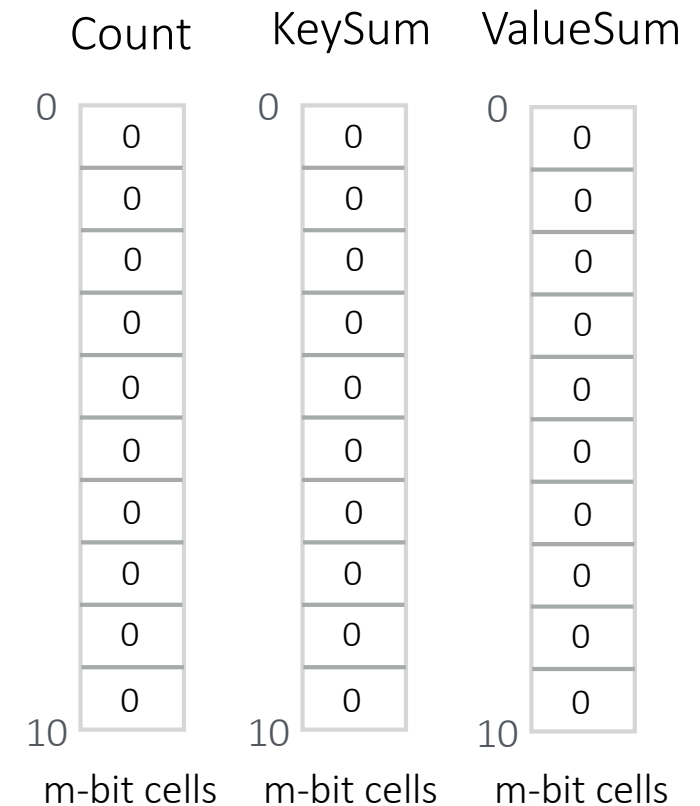
The need for something more



- Let's assume we have the Counting Bloom Filter on the left
- How can you know which elements are actually stored there?

Invertible Bloom Lookup Tables (IBLT)

- It stores {key,value} pairs and allows for lookups and listing
- Each cell contains three fields:
 - Count: counts the number of entries mapped to this cell
 - KeySum: it is the sum of all the keys mapped to this cell
 - ValueSum: it is the sum of all the values mapped to this cell



Invertible Bloom Lookup Tables (IBLT)

To add a new key-value pair (assuming it is not in the set yet)

For each hash function

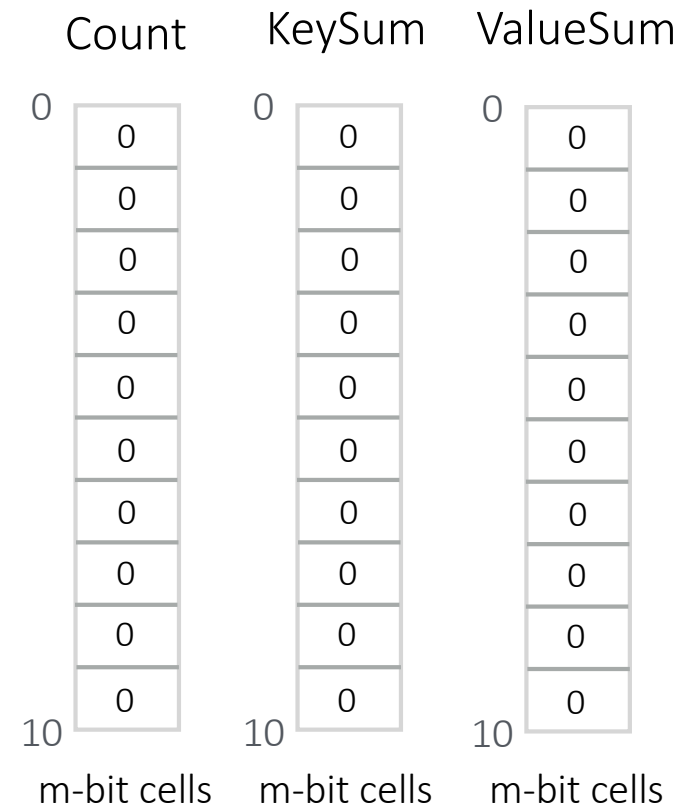
hash the key to find the index

Then at this index

increment the count by one

add key to keySum

add value to valueSum



Invertible Bloom Lookup Tables (IBLT)

To delete a key-value pair (assuming it is in the set yet)

For each hash function

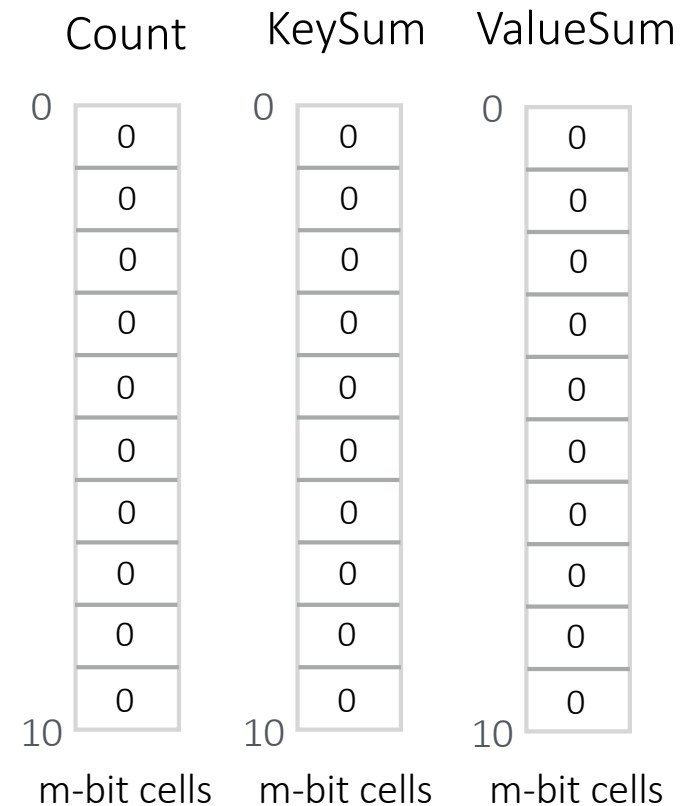
hash the key to find the index

Then at this index

subtract one to the count

subtract key to keySum

subtract value to valueSum

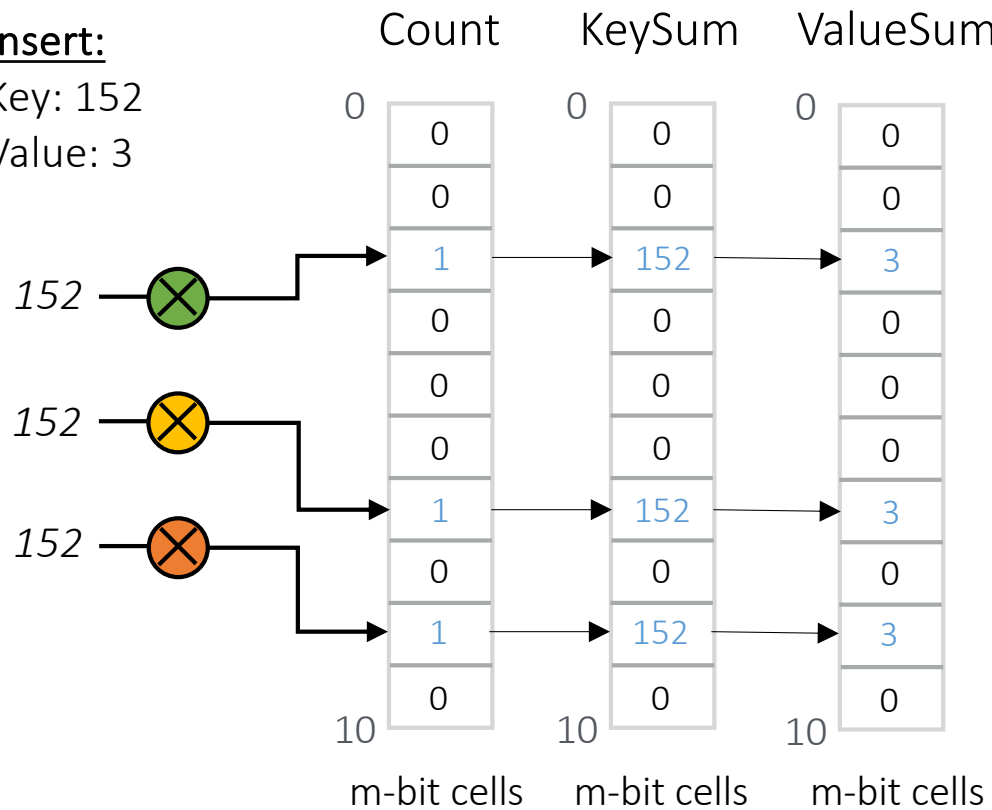


Invertible Bloom Lookup Tables (IBLT)

Insert:

Key: 152

Value: 3

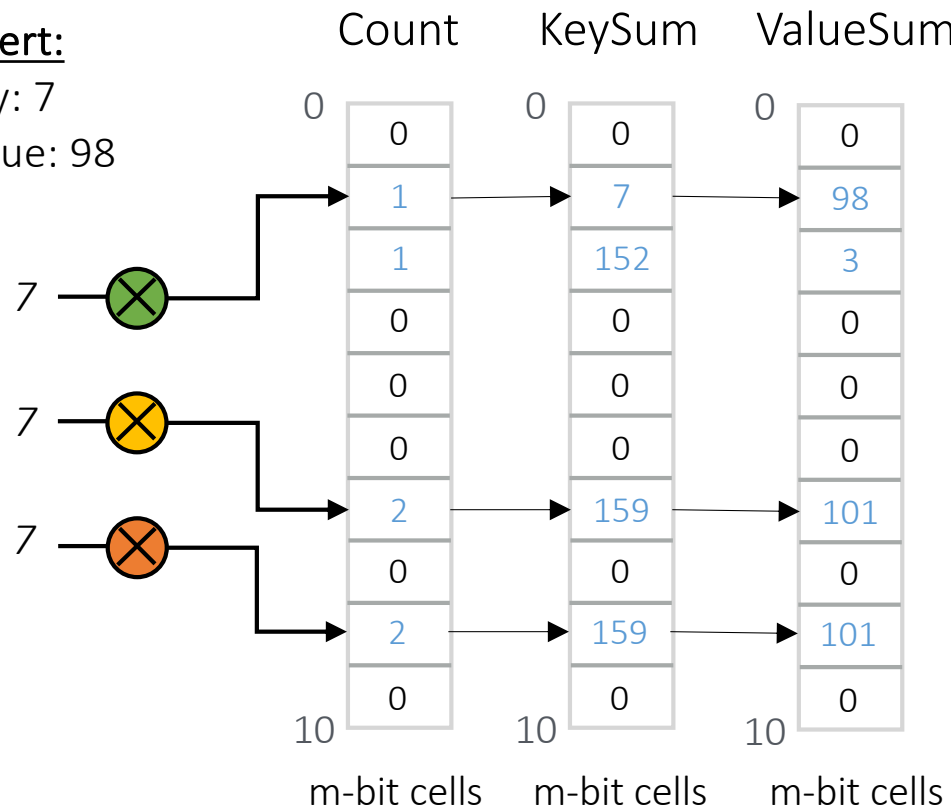


Invertible Bloom Lookup Tables (IBLT)

Insert:

Key: 7

Value: 98

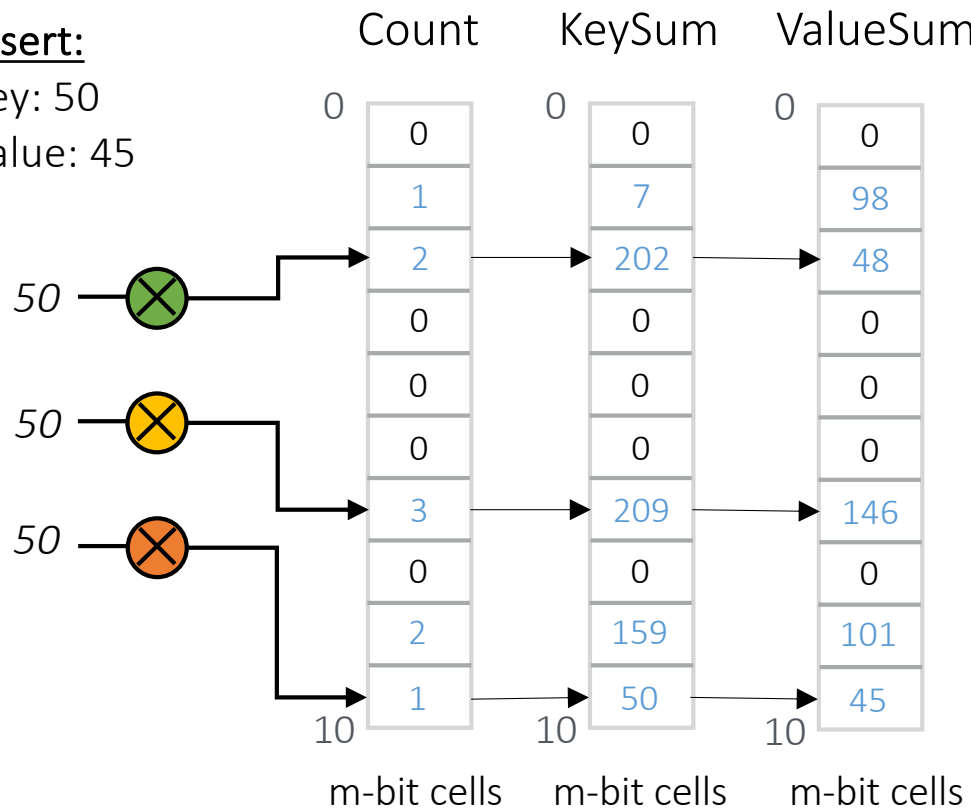


Invertible Bloom Lookup Tables (IBLT)

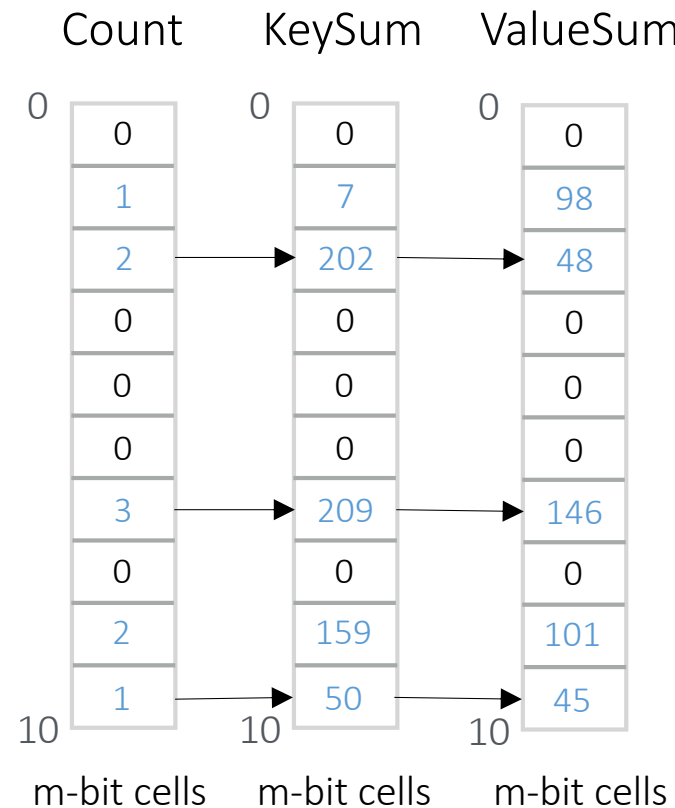
Insert:

Key: 50

Value: 45



Invertible Bloom Lookup Tables (IBLT)

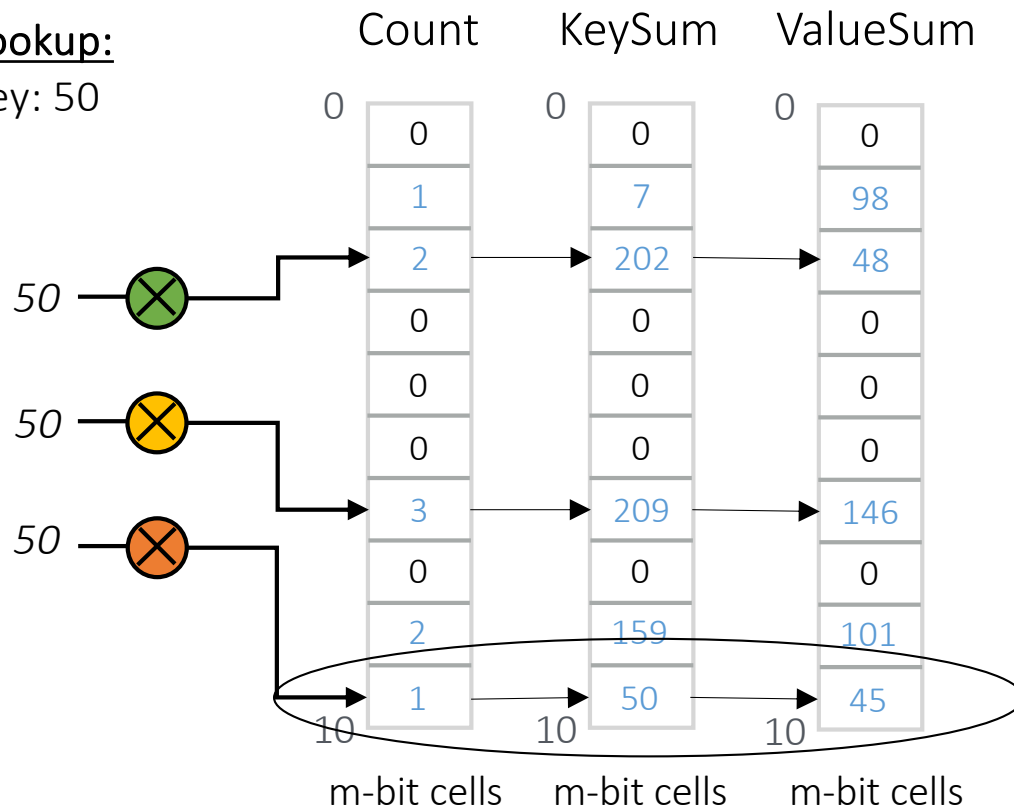


- The value of a key can be easily found if the key is associated to at least one cell with a count equals to 1

Invertible Bloom Lookup Tables (IBLT)

Lookup:

Key: 50

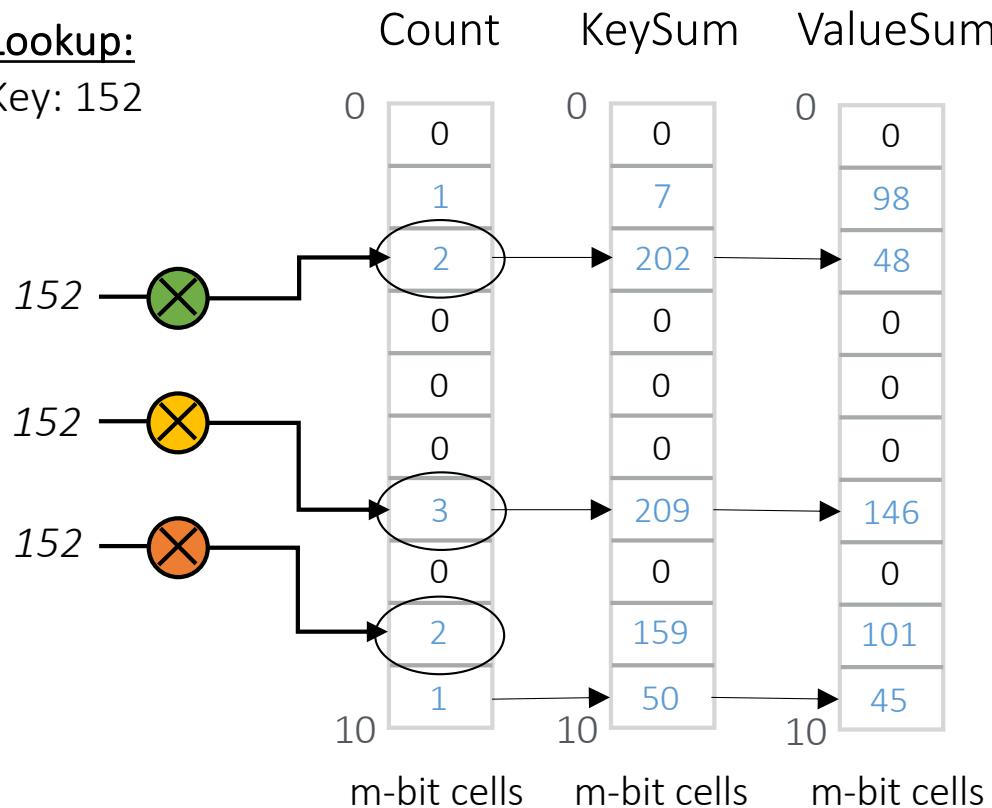


- The value of a key can be easily found if the key is associated to at least one cell with a count equals to 1
- Key 50 is associated to value 45

Invertible Bloom Lookup Tables (IBLT)

Lookup:

Key: 152



- The value of a key can be easily found if the key is associated to at least one cell with a count equals to 1
- The value associated to key 152 cannot be found (easily)

Invertible Bloom Lookup Tables (IBLT)

Count	KeySum	ValueSum
0	0	0
1	7	98
2	202	48
0	0	0
0	0	0
0	0	0
3	209	146
0	0	0
2	159	101
1	50	45
10	10	10

m-bit cells m-bit cells m-bit cells

- Listing with IBLT

While there is an index for which count = 1

Find the corresponding key-value pair and return it

Delete the corresponding key-value pair

Invertible Bloom Lookup Tables (IBLT)

Count	KeySum	ValueSum
0	0	0
1	7	98
2	202	48
0	0	0
0	0	0
0	0	0
3	209	146
0	0	0
2	159	101
1	50	45

10 m-bit cells 10 m-bit cells 10 m-bit cells

- Listing with IBLT
- So far we retrieved {}
- Let's pick the first row with count = 1. We now have that key,value pair and we can remove it from the set

Invertible Bloom Lookup Tables (IBLT)

Count	KeySum	ValueSum
0	0	0
0	0	0
2	202	48
0	0	0
0	0	0
0	0	0
2	202	48
0	0	0
1	152	3
1	50	45
10	10	10

m-bit cells m-bit cells m-bit cells

- Listing with IBLT
- So far we retrieved {(7,98)}
- Let's pick the first row with count = 1. We now have that key,value pair and we can remove it from the set
- If we delete key 7 that has value 98..

Invertible Bloom Lookup Tables (IBLT)

Count	KeySum	ValueSum
0	0	0
0	0	0
2	202	48
0	0	0
0	0	0
0	0	0
2	202	48
0	0	0
1	152	3
1	50	45
10	10	10
m-bit cells	m-bit cells	m-bit cells

- Listing with IBLT
- So far we retrieved {(7,98)}
- Let's pick another row with count = 1. We now have that key,value pair and we can remove it from the set

Invertible Bloom Lookup Tables (IBLT)

Count	KeySum	ValueSum
0	0	0
0	0	0
1	50	45
0	0	0
0	0	0
0	0	0
1	50	45
0	0	0
0	0	0
1	50	45
10	10	10
m-bit cells	m-bit cells	m-bit cells

- Listing with IBLT
- So far we retrieved {(7,98), (152,3)}
- Let's pick another row with count = 1. We now have that key,value pair and we can remove it from the set
- If we delete key 152 that has value 3..

Invertible Bloom Lookup Tables (IBLT)

Count	KeySum	ValueSum
0	0	0
0	0	0
1	50	45
0	0	0
0	0	0
0	0	0
1	50	45
0	0	0
0	0	0
1	50	45
10	10	10

m-bit cells m-bit cells m-bit cells

- Listing with IBLT
- So far we retrieved {(7,98), (152,3)}
- Let's pick another row with count = 1. We now have that key,value pair and we can remove it from the set
- Now we can easily retrieve the last one which is (50,45)

Invertible Bloom Lookup Tables (IBLT)

Count	KeySum	ValueSum
0	0	0
2	202	48
0	0	0
0	0	0
0	0	0
0	0	0
2	202	48
0	0	0
2	202	48
0	0	0
10	10	10
m-bit cells	m-bit cells	m-bit cells

- What if we end up in a situation like this one?

Invertible Bloom Lookup Tables (IBLT)

Count	KeySum	ValueSum
0	0	0
2	202	48
0	0	0
0	0	0
0	0	0
0	0	0
2	202	48
0	0	0
2	202	48
0	0	0
10	10	10
m-bit cells	m-bit cells	m-bit cells

- What if we end up in a situation like this one?
- Then, it is not possible to complete the listing 😊

Takeaways

- (Counting) Bloom Filters are very compact data structures that allow you to quickly understand if an element is in the set
 - Think about you want to implement an Intrusion Detection System and you want to redirect only specific traffic to a special firewall
 - Could you have used TCAM in this case?
- They are very efficient and allow for false positive

Counting

- Most of the time, you might want to count how often an element occurs in a data stream
 - For example, you might want to keep flow counters to understand if there are flows that are *heavier* than others
- A possible solution is to keep one counter per flow 😊
 - This does not scale well 😞 (remember switches do not have lot of memory)

Probabilistic Data Structures to the rescue

Bloom Filters

quickly filter only those elements that
might be in the set

more efficient by allowing false positives

Probabilistic Data Structures to the rescue

Bloom Filters

quickly filter only those elements that might be in the set

more efficient by allowing false positives

Sketches

provide an approximate frequencies of elements in a data stream

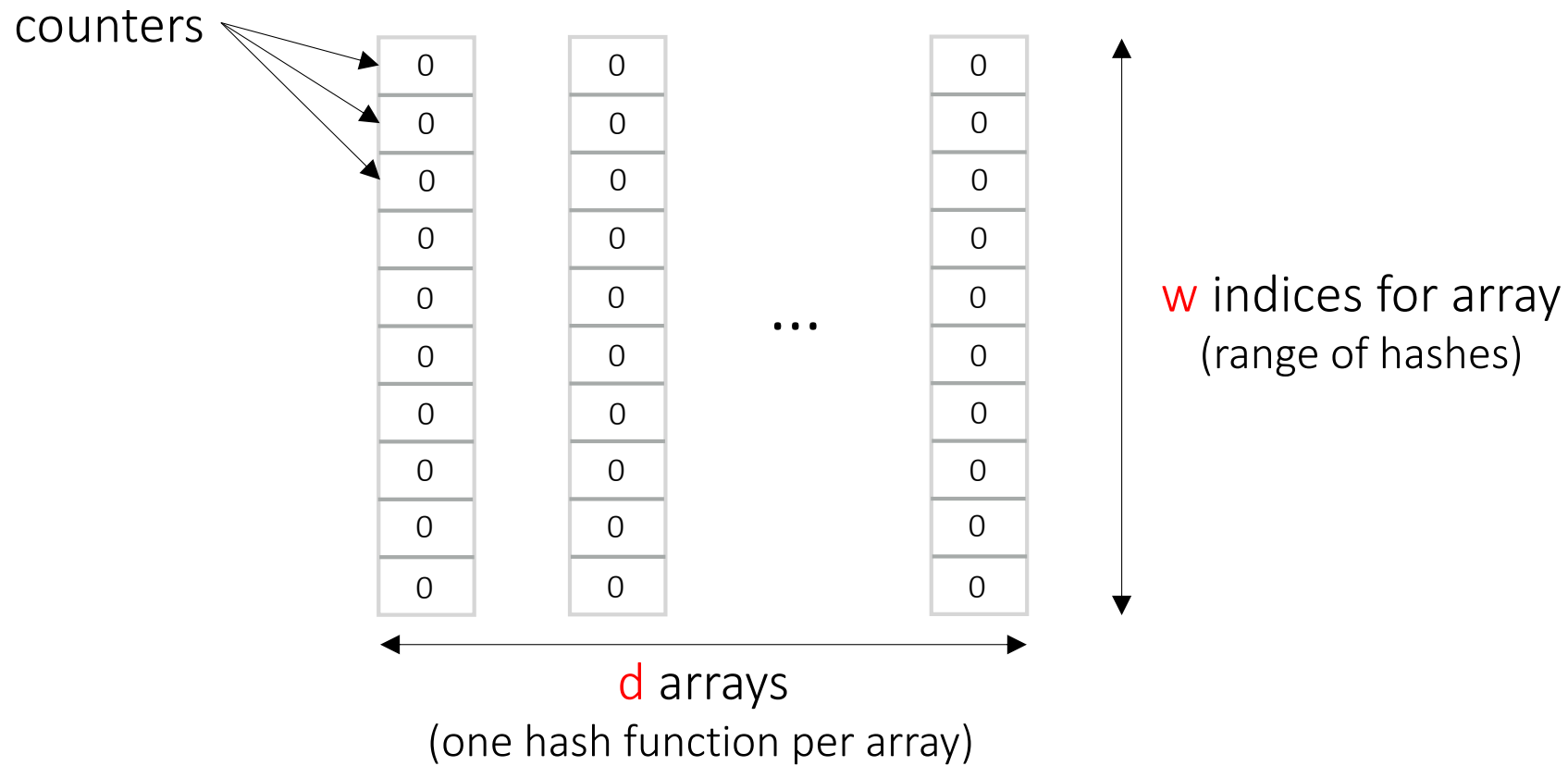
more efficient by allowing mis-counting

Count-Min Sketch

- It is a probabilistic data structure that serves as a frequency table of events in a stream of data (packets)
- It uses hash functions to map events to frequencies

Count-Min Sketch

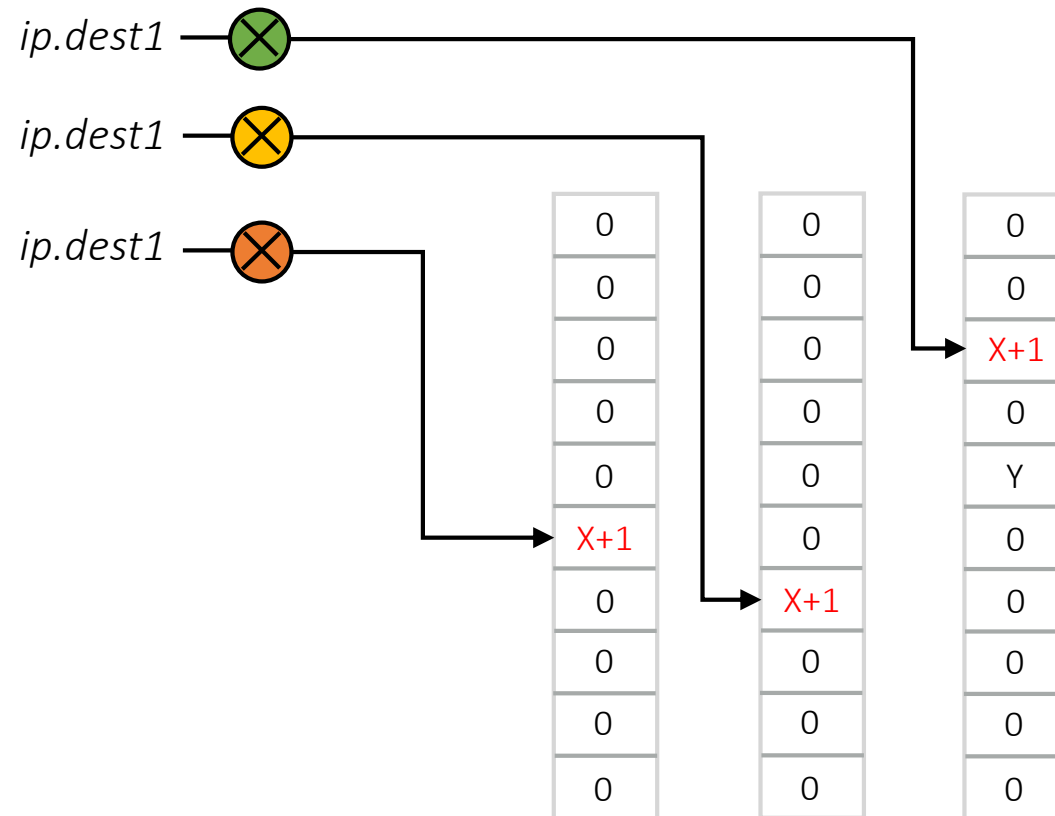
The total size is $w \times d$ counters



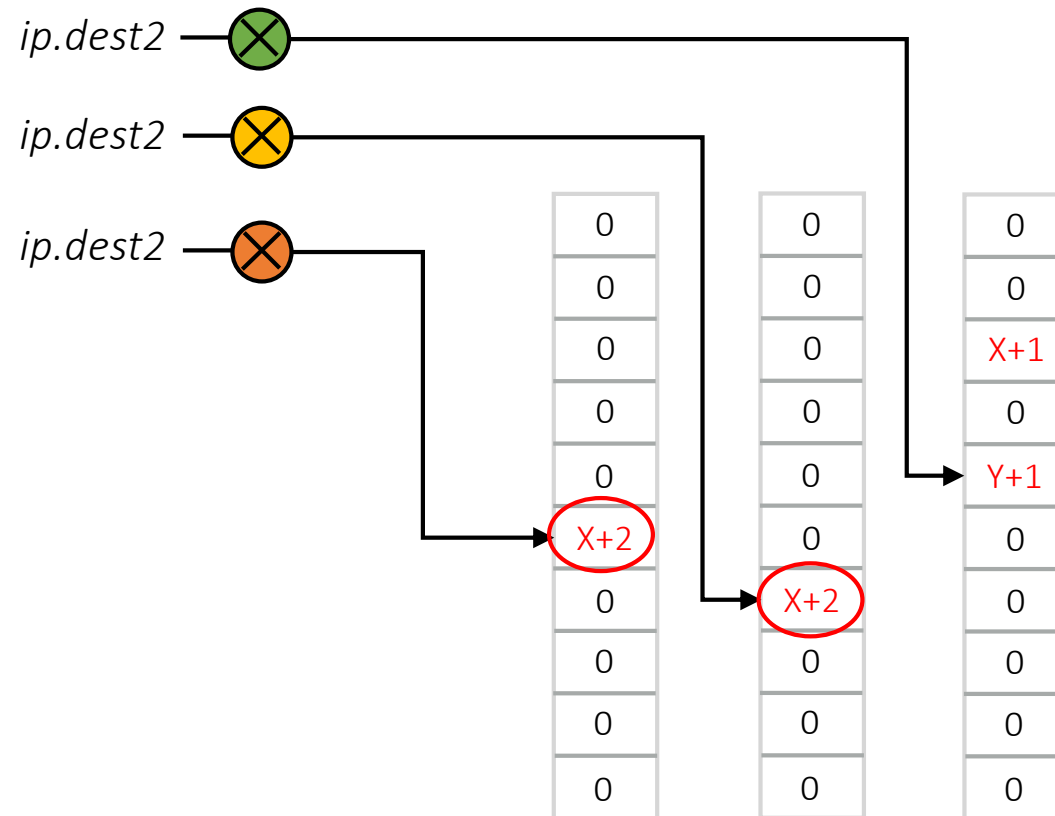
Count-Min Sketch

0	0	0
0	0	0
0	0	X
0	0	0
0	0	Y
X	0	0
0	X	0
0	0	0
0	0	0
0	0	0

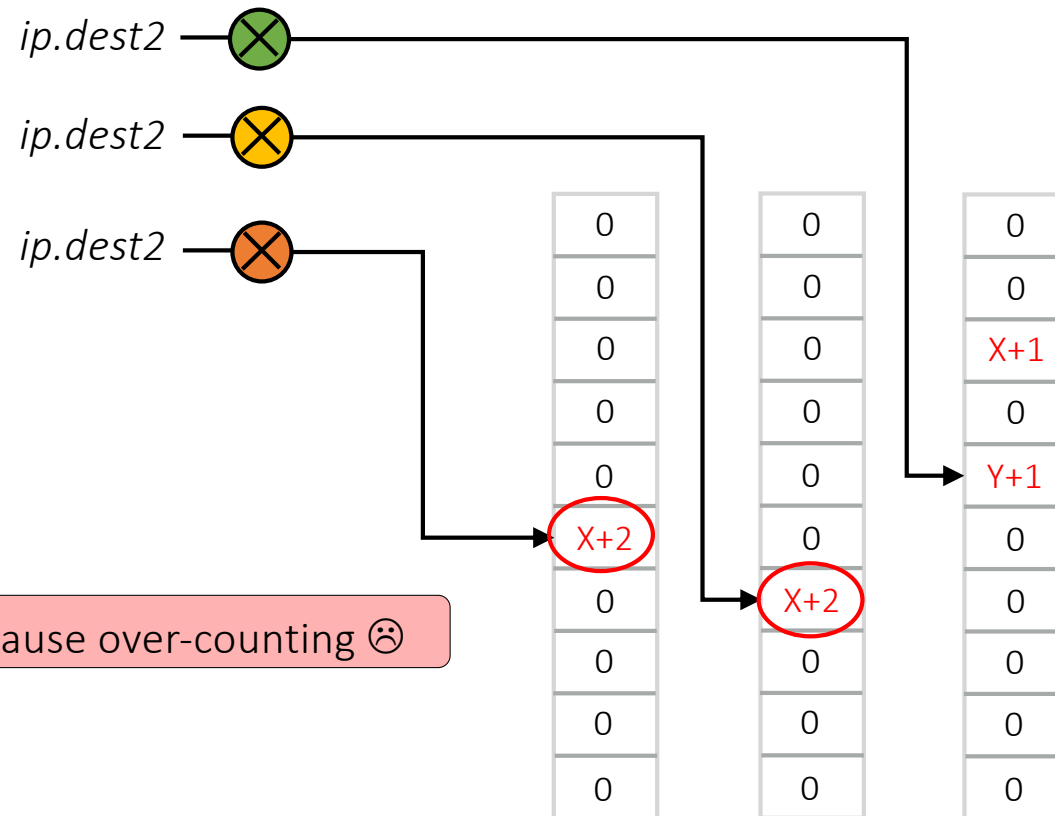
Count-Min Sketch



Count-Min Sketch

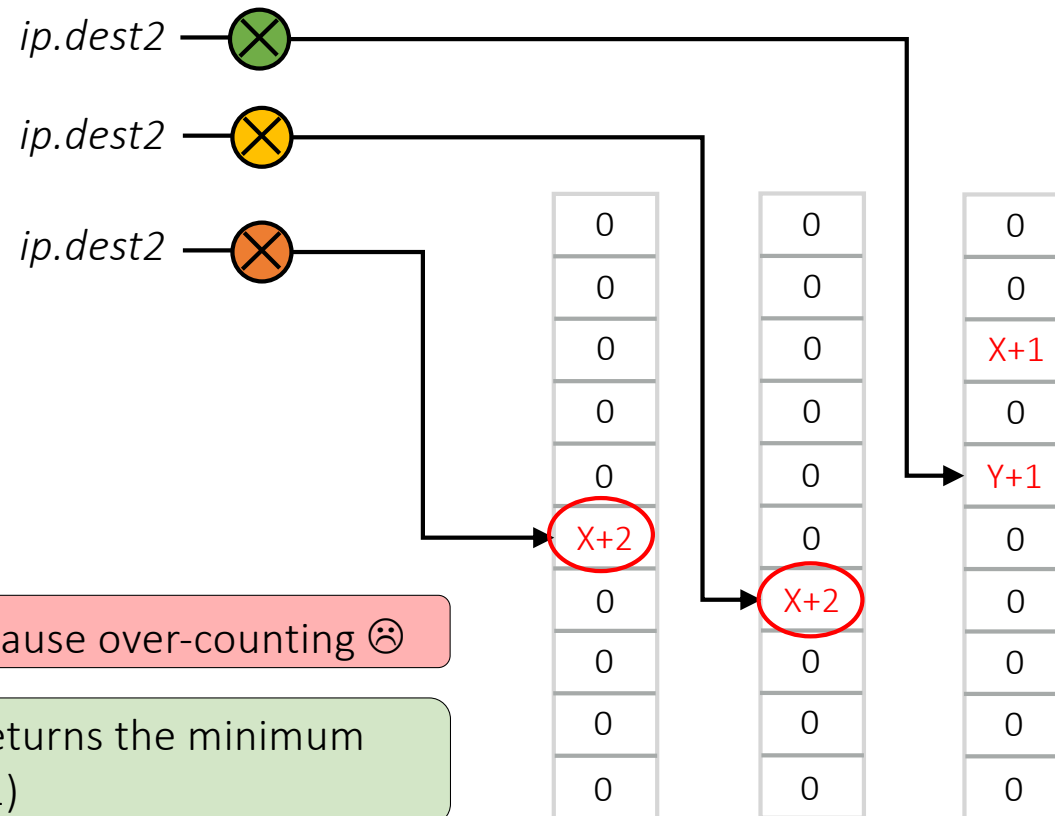


Count-Min Sketch



hash collisions can cause over-counting ☹️

Count-Min Sketch



hash collisions can cause over-counting ☹️

Count-Min Sketch returns the minimum value: $\min(X+2, Y+1)$

Count-Min Sketch

- It uses similar principles as counting bloom filter but it is designed to have provable bounds for frequency queries
- It requires sub-linear space (unlike hash tables)
 - The memory requirements do not grow linearly with the amount of data to be tracked
- Count-Min Sketch is just one example of sketch. There are plenty of them with different characteristics
- Sketches have limitations. They compute statistical summaries and favour elements with high frequency

Takeaways

- TCAM and SRAM are precious resources in switches
- We need to find ways to use them as less as possible
- (Counting) Bloom Filters allow to query if an element is present in a database
- Sketches allow to estimate the frequency of elements